

■ BEST Iași • Workshop 2026

Training Web

De la zero la site live in 2 ore (hopefully)

HTML • CSS • JavaScript • Deploy • SEO • Lighthouse

Suport pentru participanti – 6 faze, fiecare cu aplicatie practica



Faza 1 | HTML – structura, semantica, accesibilitate



Faza 2 | CSS – stilizare, layout, responsivitate



Faza 3 | JavaScript – interactivitate, manipulare DOM



Faza 4 | Deploy – versionare git, hosting GitHub Pages



Faza 5 | SEO + Open Graph – metadate pentru cautare si retele sociale



Faza 6 | Lighthouse – audit automatizat, Core Web Vitals



Resurse & urmatorii pasi – aplicatii interactive, tutoriale, comunitati

Pachetul servește atât ca suport în timpul training-ului, cât și ca referință de aprofundare ulterioară.

Structura manualului

Manualul prezent contine 7 capitole tematice (sase faze ale training-ului plus un capitol final cu resurse), structurate uniform pentru consultare facila. Fiecare capitol primeste o culoare distincta, vizibila in bara verticala stanga a blocurilor de cod si in numerotarea sectiunilor.

STRUCTURA UNUI CAPITOL

- **Teorie sintetica** – concepte, sintaxa, referinte la standarde.
- **Bloc de cod** – fragmente reutilizabile direct prin copiere.
-  **Aplicatie practica** – platforma online cu sarcina de exersare (5–10 minute).
-  **Resurse pentru aprofundare** – bibliografie cu linkuri.

CONVENTII VIZUALE

- Blocurile de cod sunt marcate printr-o bara verticala colorata in stanga, in culoarea capitolului.
- Toate URL-urile sunt clicabile in vizualizatorul PDF.
- Iconitele       identifica vizual fiecare capitol.

Proiectul fir roșu

Construim împreuna portofoliul Voluntarului BEST.

Manualul nu prezinta concepte izolate. Pe parcursul celor 6 faze, fiecare strat se adauga la o pagina concreta – portofoliul Voluntarului BEST, studenta CS in Iasi. La final, fiecare participant aplica acelasi pattern pentru pagina sa proprie.

Evoluția paginii pe parcursul training-ului:

Faza 1 – HTML. Text negru pe fundal alb, ca un document Word minimal. Structura semantica corecta in spate.

Pagina *exista* la nivel de informație, dar arata generic.

+ Faza 2 – CSS. Pagina capata identitate vizuala: paleta de culori, layout cu flexbox, header cu logo și navigare, hero cu CTA, dark mode.

Pagina *arata* ca un site profesional. Nu raspunde la interacțiuni.

+ Faza 3 – JavaScript. Hamburger menu funcțional pe mobil, smooth scroll la link-uri, formular care nu reincarca pagina.

Pagina *raspunde* la utilizator. Ramane offline, accesibila doar local.

+ Faza 4 – Deploy. Site-ul devine accesibil la voluntar-best.github.io/portfolio. URL real, share-uibil oriunde.

Pagina *exista in lume*. Dar oamenii inca nu o pot gasi.

+ Faza 5 – SEO. Cand cineva paste-uieste link-ul pe WhatsApp, apare card cu titlu, descriere si imagine. Google indexeaza pagina corect.

Pagina *e descoperibila*. Tehnic poate fi inca rafinata.

+ Faza 6 – Lighthouse. Audit pe 4 categorii. Scor 95+ pe Performance, Accessibility, Best Practices, SEO.

Pagina *e profesionala*. Standardul pe care firmele il cer la angajare.

✔ PE SCURT

Fiecare faza adauga o capacitate noua paginii. La final, pagina ta personala (cu numele tau in URL) trece toate verificarile pe care un dezvoltator profesional le-ar face.

Cuprins

Capitole, în ordinea recomandată.

Faza 1 – HTML	4
Faza 2 – CSS	15
Faza 3 – JavaScript	34
Faza 4 – Deploy	48
Faza 5 – SEO + Open Graph	54
Faza 6 – Lighthouse	64
Resurse & următorii pași	74
Anexe – Glosar, Checklist, Erori comune	79

Faza 1 – HTML

Structura documentului. Tag-uri semantice. Accesibilitate.

HTML (*HyperText Markup Language*) este limbajul de marcare folosit pentru descrierea **structurii** unui document web. Browserul citește documentul de sus în jos, construiește în memorie un arbore de noduri (DOM-ul), și afișează pagina pe ecran.

În arhitectura unei pagini web, responsabilitățile sunt împartite între trei tehnologii:

- **HTML** – ce conține pagina (titluri, paragrafe, imagini, formulare).
- **CSS** – cum arată pagina (culori, dimensiuni, spațiere, layout).
- **JavaScript** – cum se comportă pagina (răspuns la click, animații, validare).

Capitolul definește anatomia minimală, tag-urile semantice principale, atributele obligatorii pentru accesibilitate, și patternurile uzuale pentru linkuri și formulare.

🕒 CE ÎNVEȚI

După capitolul asta vei ști:

- Scrie boilerplate-ul HTML5 corect, cu cele 3 meta tags obligatorii.
- Distinge tag-urile semantice de `<div>` și alege tag-ul potrivit per zonă.
- Construiește formulare accesibile cu `<label for>` și `<input id>`.
- Aplică atributele `alt`, `lang`, `href` cu valori corecte conform WCAG.
- Recunoaște pattern-urile pentru linkuri externe sigure (`rel="noopener"`).
- 🧠 *Bonus self-study:* `<picture>` cu fallback de format, `loading="lazy"` + `decoding="async"`, ARIA (`aria-label`, `aria-current`).

❓ DE CE

HTML este temelia oricărei pagini web. Browserele, cititoarele de ecran și motoarele de căutare interpretează pagina pornind de la structura HTML. O structură semantică greșită conduce la trei consecințe directe: SEO penalizat, persoanele cu dizabilități nu pot folosi pagina, codul devine ilizibil după câteva luni. Toate fazele următoare (CSS, JS, deploy, audit) presupun că structura HTML este corectă.

1. Anatomia documentului

Un document HTML5 conține cinci elemente structurale obligatorii. Pe linia 1, declarația `<!DOCTYPE html>` indică browserului că documentul respectă specificația HTML5; în absența sa, browserul intră în modul „quirks” care reproduce comportamentul browserelor din anii 2000.

Listing 1: Boilerplate HTML5 conform specificației W3C

```
1 <!DOCTYPE html>
2 <html lang="ro">
3 <head>
4   <meta charset="UTF-8">
```

```

5   <meta name="viewport" content="width=device-width,
      initial-scale=1.0">
6   <title>Numele Prenumele -- Student CS, Iasi</title>
7 </head>
8 <body>
9   <!-- continut vizibil -->
10 </body>
11 </html>

```

- `<html lang="ro">` – elementul radacina. Atributul `lang` influenteaza pronuntia in cititoare de ecran, ranking-ul in cautari localizate, si detectia limbii sursa pentru traducere automata.
- `<head>` – contine metadatele paginii. Continutul nu apare pe ecran, dar este consumat de browser, motoare de cautare si platforme sociale.
- `<body>` – contine elementele vizibile pe ecran. Tot ce vede utilizatorul se afla in interiorul acestui tag.

2. Sintaxa tag-urilor

Un element HTML are una sau mai multe parti: tag de deschidere, continut, tag de inchidere. Atributele se declara in tag-ul de deschidere, in formatul `nume="valoare"`.

Listing 2: Tipuri de tag-uri

```

1 <!-- Tag container: are continut intre deschidere si inchidere -->
2 <p>Acesta este un paragraf.</p>
3
4 <!-- Tag void: nu are continut, nu se inchide -->
5 
6
7 <!-- Tag cu attribute multiple -->
8 <a href="https://best-is.ro" target="_blank" rel="noopener">BEST</a>

```

Tag-urile void cuprind: `<img>`, `<br>`, `<hr>`, `<input>`, `<meta>`, `<link>`.

3. Metadate obligatorii in `<head>`

Trei elemente sunt considerate obligatorii in orice pagina moderna:

1. `<meta charset="UTF-8">` – setul de caractere. UTF-8 acopera toate alfabetele moderne, inclusiv diacriticele romanesti.
2. `<meta name="viewport"...>` – controleaza redarea pe dispozitive mobile. Valoarea `width=device-width, initial-scale=1.0` instruceaza browserul sa pastreze latimea paginii egala cu cea a ecranului si sa nu zoom-eze initial.
3. `<title>` – apare in tab-ul browserului, in bookmark-uri, si ca titlu in rezultatele cautarii.

⚠ ATENȚIE

Conform datelor StatCounter (2024), peste 60% din traficul web global provine de pe dispozitive

mobile. Lipsa meta tag-ului `viewport` afectează direct experiența majorității utilizatorilor, generând o pagină nescălată, cu zoom necesar pentru lectură.

4. Tag-uri semantice vs `<div>` soup

Tag-urile semantice descriu *rolul* conținutului, nu doar aspectul vizual. Browserul, motoarele de căutare și tehnologiile asistive (cititoare de ecran, navigatoare prin tastatură) pot interpreta automat structura paginii când tag-urile semantice sunt folosite.

✗ ASA NU – „DIV SOUP”

```
<div class="header">
  <div class="logo">Site</div>
  <div class="menu">
    <div class="link"><a href="/">Acasa</a></div>
  </div>
</div>
<div class="content">
  <div class="hero">
    <div class="title">Salut</div>
  </div>
</div>
<div class="footer">2026</div>
```

Probleme. Browserul și cititoarele de ecran nu știu ce reprezintă fiecare `<div>`. Navigația rapidă prin landmarks este imposibilă. SEO penalizat pentru lipsa de structură.

✓ ASA DA – SEMANTIC

```
<header>
  <a class="logo" href="/">Site</a>
  <nav>
    <a href="/">Acasa</a>
  </nav>
</header>
<main>
  <section class="hero">
    <h1>Salut</h1>
  </section>
</main>
<footer>2026</footer>
```

Beneficii. Cititoarele de ecran oferă comenzi rapide pentru sărit la `<header>`, `<nav>`, `<main>`, `<footer>`. Google identifică zonele principale fără analiză euristică. Codul este mai ușor de citit după 6 luni.

4.1 Repertoriul tag-urilor semantice

- `<header>` – antetul unei pagini sau al unei secțiuni.
- `<nav>` – bara de navigație principală.

- `<main>` – conținutul unic al paginii (un singur `<main>` per pagina).
- `<section>` – grup tematic de conținut, de obicei cu un `<h2>`.
- `<article>` – unitate de conținut auto-suficientă (post de blog, comentariu, card produs).
- `<aside>` – conținut tangential (sidebar, note de subsol vizuale).
- `<footer>` – subsolul unei pagini sau al unei secțiuni.
- `<figure>` + `<figcaption>` – imagine cu legenda.

5. Ierarhia titlurilor

Specificația HTML5 definește șase niveluri de titluri, de la `<h1>` la `<h6>`. Cititoarele de ecran generează un *outline* al paginii pe baza acestei ierarhii; un utilizator nevăzător parcurge documentul saltând între titluri exact ca un cititor obișnuit când parcurge un cuprins.

- Un singur `<h1>` per pagina (convenția industrială pentru accesibilitate și SEO; HTML Living Standard permite tehnic mai multe, dar Lighthouse și cititoarele de ecran așteaptă unul singur), descriind tema centrală.
- Subsecțiunile principale folosesc `<h2>`; sub-subsecțiunile, `<h3>`.
- Nivelurile nu se sar (`<h1>` urmat direct de `<h3>` este o eroare structurală).
- Marimea vizuală se controlează prin CSS, nu prin alegerea unui nivel diferit.

✗ ASA NU – STILUL ALES PRIN NIVEL

```
<h1>Titlul mare</h1>
<h4>Subtitlu, am ales h4 ca arata mai mic</h4>
<h2>Secțiune</h2>
```

✓ ASA DA – STILUL ALES PRIN CSS

```
<h1>Titlul mare</h1>
<h2 class="subtitle">Subtitlu</h2>
<h2>Secțiune</h2>
```

```
.subtitle { font-size: 1rem; font-weight: normal; }
```

6. Linkuri și navigare

Elementul `<a>` (anchor) creează un hyperlink. Atributul `href` acceptă mai multe forme:

Listing 3: Tipuri de URL-uri

```
1 <!-- URL absolut -->
2 <a href="https://best-is.ro">BEST Iasi</a>
3
4 <!-- URL relativ -->
5 <a href="contact.html">Contact</a>
```



```

6 <a href="../../../images/logo.png">Logo</a>
7
8 <!-- Ancora interna -->
9 <a href="#contact">Sari la contact</a>
10
11 <!-- Adresa email / numar de telefon -->
12 <a href="mailto:hello@best-is.ro">Email</a>
13 <a href="tel:+40123456789">Telefon</a>

```

6.1 Linkuri externe – target si rel

Deschiderea unui link extern intr-un tab nou se face cu `target="_blank"`. Fara protectie, pagina destinatie poate manipula tab-ul sursa prin `window.opener` (tabnabbing → phishing). Adauga mereu `rel="noopener noreferrer"`:

```

<a href="https://extern.ro" target="_blank" rel="noopener
  noreferrer">Extern</a>

```

`noopener` blocheaza accesul la `window.opener`; `noreferrer` ascunde URL-ul sursa.

7. Formulare accesibile

Fiecare element `<input>` trebuie asociat cu un `<label>`. Asocierea se realizeaza prin perechea atribut-valoare `for/id`: valoarea atributului `for` de pe `<label>` este identica cu valoarea atributului `id` de pe `<input>`.

✗ ASA NU – PLACEHOLDER CA SUBSTITUT PENTRU LABEL

```

<input type="email" placeholder="Email">

```

Probleme. (1) Cititoarele de ecran nu anunta placeholder-ul ca eticheta. (2) Placeholder-ul dispare la tastare, lasand utilizatorul fara context. (3) Contrast scazut pe textul de placeholder.

✓ ASA DA

```

<label for="email">Email</label>
<input type="email" id="email" name="email" required>

```

Beneficii. Cititoarele anunta corect; clic-ul pe text focuseaza inputul; eticheta ramane vizibila.

7.1 Tipuri de input frecvente

Atributul `type` controleaza interfata oferita de browser si validarea. Pe mobil, tipul determina tastatura afisata.

- `type="text"` – text generic.
- `type="email"` – validare format email; tastatura cu @ pe mobil.
- `type="tel"` – tastatura numerica pe mobil; nu valideaza formatul (variaza pe tari).
- `type="number"` – accepta doar cifre; arrows pe desktop.

- `type="date"` – date picker nativ.
- `type="password"` – ascunde caracterele.
- `type="url"` – validare format URL.
- `type="search"` – buton clear in unele browsere.

🔗 PONT

Browserul valideaza nativ inputurile: campurile cu atributul `required` blocheaza submit-ul daca sunt goale; `type="email"` respinge valori care nu contin @. Aceasta validare **nu inlocuieste** validarea pe server, dar imbunatateste experienta utilizatorului inainte de a trimite cererea.

7.2 Element HTML – rol UI generic

Daca ai mai folosit o interfata grafica (QtCreator, Figma, Visual Studio, Android Studio), recunosti aceste pattern-uri sub alte nume. Tabelul de mai jos pune in oglinda terminologia:

Element HTML

```
<label>
<input type="text">
<input type="email">
<input type="password">
<input type="number">
<input type="date">
<input type="range">
<textarea>
<select> + <option>
<input type="radio">
<input type="checkbox">
<input type="file">
<input type="color">
<button>
<a href="...">
<img>
<video> / <audio>
<details> + <summary>
<dialog>
<progress>
```

Rolul UI generic

```
text static (eticheta unui camp)
lineedit – input pentru text scurt
lineedit cu validare email
lineedit ascuns (parola)
spinbox – numar cu sageti
date picker
slider – valoare pe interval
text multi-linie
combobox – o alegere dintr-o lista
radio button – o singura alegere dintr-un grup
checkbox – alegeri multiple (sau pornit/oprit)
file dialog – selectie fisier
color picker
buton de actiune
link – navigare catre URL
image statica
player media nativ
accordeon – arata/ascunde
modal – fereastra suprapusa
bara de progres
```

Regula: cand alegi un element, te intrebi mai intai „ce rol UI joaca?” – nu „cum ar arata cu CSS”. Tag-ul corect rezolva accesibilitatea, validarea și comportamentul nativ *gratis*.

8. Atribute esentiale

- `href` pe `<a>` – URL-ul tinta.
- `src` pe `<img>` – calea catre imagine.
- `alt` pe `<img>` – text descriptiv. Obligativu conform WCAG 2.1 (criteriul 1.1.1 *Non-text Content*).
- `type` pe `<input>` – determina validarea si interfata.
- `required` pe `<input>` – activeaza validarea nativa.
- `lang` pe `<html>` – limba documentului.
- `title` pe orice element – tooltip la hover (uz limitat, nu inlocuieste `aria-label`).

8.1 Calitatea textului alt

Textul `alt` descrie ce vede imaginea, in contextul paginii. Scopul: utilizatorul fara acces vizual primeste informatia echivalenta.

✗ ASA NU – ALT NESEMNIFICATIV SAU SPAM SEO

```



```

✓ ASA DA – ALT DESCRIPTIV SI SPECIFIC

```

```

Exceptie: pentru imagini pur decorative (fundal, separator, ornament), `alt=""` este corect – cititorul de ecran le ignora.

9. Tag-uri suplimentare – mențiuni rapide

Pentru proiecte mai complexe, HTML5 ofera elemente specializate care nu intra in scopul fazei curente, dar merita cunoscute la nivel de existența:

- `<table>` cu `<thead>`, `<tbody>`, `<tfoot>`, `<caption>` – pentru date tabulare reale (nu pentru layout). Atributele `scope`, `headers` asigura accesibilitatea.
- `<video>` si `<audio>` – mediafile native, fara nevoie de plugin-uri (Flash, Silverlight au murit). Atribute: `controls`, `autoplay`, `muted`, `loop`, `poster` (pentru video).
- `<canvas>` – suprafata 2D pentru desenare programatica cu JavaScript. Folosit pentru jocuri, vizualizari, animatii complexe.
- `<details>` si `<summary>` – accordeon nativ, fara JavaScript.
- `<dialog>` – modal nativ, suportat in toate browserele moderne.

- `<picture>` – container pentru `<img>` cu surse alternative pe formate sau dimensiuni.

Documentația exhaustivă: MDN – HTML elements reference (link în resursele de la finalul capitoului).

10. Comentarii și caractere speciale

Comentariile HTML nu apar în pagina dar rămân vizibile în View Source. Sintaxa: `<!-- text -->`.

Caracterele rezervate (`<`, `>`, `&`) trebuie scrise prin entități pentru a nu fi interpretate ca markup:

- `<` – `<`
- `>` – `>`
- `&` – `&`
- `©` – (c)
- ` ` – spațiu nedespartibil

🔧 BONUS • AVANSAT

Acest box e optional. Dacă ești la început, treci direct la Verificare. Pattern-urile de mai jos pregătesc terenul pentru Lighthouse și accesibilitate avansată.

📧 `<picture>` – imagini responsive cu fallback

`<img>` obișnuit servește o singură sursă. `<picture>` alege între mai multe surse: format modern (AVIF → WebP → JPEG fallback) sau dimensiune (mobil vs desktop):

```
<picture>
  <source srcset="hero.avif" type="image/avif">
  <source srcset="hero.webp" type="image/webp">
  
</picture>
```

Browserul încarcă **prima sursă pe care o susține**. Chrome modern → AVIF (cel mai mic), Safari vechi → JPEG. Zero JavaScript, zero detecție de feature.

⚡ `loading="lazy"` și `decoding="async"`

Două atribute care îmbunătățesc semnificativ Lighthouse fără cost:

```

  decoding="async"        <!-- decodifica pe alt thread, nu
    blocheaza UI -->
  width="200" height="200">
```

Excepție: imaginea hero (prima vizibilă) nu primește `loading="lazy"` – ar întârzia LCP. Pentru ea: `fetchpriority="high"`.

♿ ARIA – când semantica nu e suficientă

ARIA (Accessible Rich Internet Applications) adaugă semnificație pentru cititoarele de ecran când HTML-ul semantic nu acoperă contextul. **Regula de aur:** prima alegere e mereu un tag semantic.

continuare bonus

ARIA il completeaza.

Cele 3 atribute pe care le folosești des:

- `aria-label="..."` – eticheta accesibila cand textul vizibil lipseste sau e ambiguu. Exemplu clasic: butonul hamburger.

```
<button class="menu-toggle" aria-label="Deschide meniul">
  <svg>...</svg>
</button>
```

- `aria-current="page"` – indica link-ul activ in navigare:

```
<nav>
  <a href="/" aria-current="page">Acasa</a>
  <a href="/projects">Proiecte</a>
</nav>
```

- `role="..."` – rar necesar. Folosește-l doar cand transformi un `<div>` in ceva semantic (ce nu ar trebui sa faci). Excepție utila: `role="alert"` pentru notificari live.

ATENȚIE

NU adauga `role="button"` pe un `<button>`, sau `role="navigation"` pe un `<nav>`. Tag-ul semantic are deja rolul corect. ARIA duplicat = bruiat pentru cititoarele de ecran.

Unde mergi de aici

- MDN `<picture>` reference: <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/picture>
- web.dev Lazy load images: <https://web.dev/articles/browser-level-image-lazy-loading>
- WAI-ARIA Authoring Practices: <https://www.w3.org/WAI/ARIA/apg/>

Verificare cunoștințe

PROVOCARE

1. Care este diferența funcțională între `<div class="header">` și `<header>`?
2. De ce este nevoie de `<meta name="viewport">` pentru afișarea pe mobil?
3. Spot the issue: `<input type="email" placeholder="Email">`.
4. Cate elemente `<h1>` recomanda convenția industrială (accesibilitate + SEO) sa existe intr-o pagina?
5. Care sunt cele doua atribute necesare cand un link extern se deschide intr-un tab nou?

Recapitulare

✓ FAZA 1 – HTML

- Anatomia: `<!DOCTYPE html>` → `<html lang>` → `<head>` (metadate) → `<body>` (continut).
- Trei meta tags obligatorii: `charset`, `viewport`, `<title>`.
- Tag-uri semantice in locul `<div>`: `<header>`, `<main>`, `<section>`, `<footer>`, `<nav>`, `<article>`, `<aside>`.
- Un singur `<h1>` per pagina; ierarhia titlurilor nu admite salturi de nivel.
- Forms accesibile: `<label for="X">` corespunde cu `<input id="X">`.
- Attribute obligatorii pentru accesibilitate: `alt` pe `<img>`, `lang` pe `<html>`.
- Linkuri externe in tab nou: combinatia `target="_blank" + rel="noopener noreferrer"`.
- 🦉 Bonus: `<picture>` pentru fallback AVIF/WebP/JPEG, `loading="lazy" + decoding="async"` pe imagini, ARIA (`aria-label`, `aria-current`).



🛠️ APLICAȚIE PRACTICĂ

Platforma: CodePen (codepen.io). URL-ul exact este distribuit de trainer in chat.

Activitatea. Documentul de pornire contine o structura non-semantica (`<div>` folosit excesiv), attribute `alt` lipsa pe imagini si elemente `<input>` fara `<label>` asociat. Sarcina este de a transforma documentul intr-o varianta semantica si conformanta cu standardele de accesibilitate.

Criterii de validare:

- Folosirea tag-urilor `<header>`, `<main>`, `<section>`, `<footer>` in locul `<div>`.
- Prezenta atributului `alt` pe toate elementele `<img>`.
- Asocierea `<label for>` – `<input id>` pentru fiecare camp de formular.
- Existenta unui singur element `<h1>` per pagina.
- Linkuri externe au `target="_blank" + rel="noopener noreferrer"`.

Durata estimata: 5 minute.

🎓 Resurse pentru aprofundare

- Mozilla Developer Network. *HTML elements reference*. <https://developer.mozilla.org/en-US/docs/Web/HTML/Element>
- The Odin Project. *HTML Foundations*. <https://www.theodinproject.com/paths/foundations/courses/foundations#html-foundations>
- Google. *Learn HTML*. <https://web.dev/learn/html>
- HTML Reference. *Vizual cheatsheet de tag-uri*. <https://htmlreference.io>
- Google. *Learn Forms*. <https://web.dev/learn/forms>
- W3C. *Web Accessibility Initiative – WCAG*. <https://www.w3.org/WAI/standards-guidelines/wcag/>

→ **Urmeaza:** Capitolul 2 (CSS) imbraca structura HTML cu stil vizual. Aspectul, layout-ul și responsivitatea se construiesc plecand de la pagina semantica generata aici.

Faza 2 – CSS

Stilizare. Layout. Responsivitate.

CSS (*Cascading Style Sheets*) controlează aspectul vizual al documentelor HTML. Numele provine de la *cascada*: când mai multe reguli vizează același element, browserul aplică un algoritm de prioritate (cascada) pentru a alege valoarea finală.

Capitolul prezintă selectorii, modelul de cutie, unitățile de măsură, variabilele CSS, sistemele de layout flexbox și grid, abordarea *mobile-first*, și suportul nativ pentru tema dark/light.

🎯 CE ÎNVEȚI

După capitolul asta vei ști:

- Folosește selectorii CSS și înțelege specificitatea ca triplet (`id`, `clasa`, `tag`).
- Aplica modelul de cutie cu `box-sizing: border-box` pentru calcule predictibile.
- Definește variabile CSS în `:root` și le refolosește prin `var()`.
- Construiește layout-uri cu flexbox (6 proprietăți cheie).
- Scrie reguli mobile-first cu `@media (min-width: ...)`.
- Poziționează elemente cu `position` și creează header sticky.
- Aplica animații cu `transition` și `@keyframes`, respectând `prefers-reduced-motion`.
- Combina `transform`, `transition` și `cubic-bezier()` pentru micro-interacțiuni profesionale (hover-lift, fade-in, spinner).
- 🐦 *Bonus self-study*: transformări 3D (flip-card), `animation-timeline` pentru scroll-driven, `filter/backdrop-filter`, `clip-path`, container queries.

❓ DE CE

CSS face diferența între pagina-document și pagina-produs. Aceeași structură HTML poate părea neprofesională sau profesională, în funcție de stilizare. Mai mult: CSS-ul prost arhitecturat (cu `!important`, specificitate ridicată, valori absolute) generează ore de depanare. Convențiile prezentate aici (variabile, mobile-first, flexbox) sunt cele aplicate în practica industrială.

📖 PRE-RECHIZITE

Capitolul presupune înțelegerea tag-urilor HTML semantice și a structurii `<head>/<body>` (Faza 1).

1. Conectarea fișierului CSS la HTML

CSS poate fi inclus în trei moduri. Convenția recomandată este externalizarea într-un fișier separat:

Listing 4: Stilurile externe (recomandat)

```
1 <head>
2   <link rel="stylesheet" href="style.css">
3 </head>
```


✘ ASA NU – STILURI INLINE IMPRASTIATE

```
<button style="background: blue; color: white; padding: 10px;">Click</button>
```

Probleme. Imposibil de reutilizat; stilul nu poate fi schimbat global; specificitatea ridicata creeaza conflicte; pagina HTML devine ilizibila.

✔ ASA DA – CLASA CU STIL EXTERN

```
<button class="cta">Click</button>
```

In `style.css`: `.cta { background: blue; color: white; padding: 10px; }`

2. Selectorii

Selectorii determina elementele asupra carora se aplica un set de reguli. Categoriile principale:

Listing 5: Selectorii uzuali

```
1  /* Selector de tip: vizeaza toate elementele <h1> */
2  h1 { color: navy; }
3
4  /* Selector de clasa: reutilizabil, prefixat cu . */
5  .cta { background: orange; }
6
7  /* Selector de id: unic pe pagina, prefixat cu # */
8  #contact { padding: 2rem; }
9
10 /* Selector de atribut */
11 input[type="email"] { border-color: blue; }
12
13 /* Combinator descendent: <a> oriunde sub <nav> */
14 nav a { text-decoration: none; }
15
16 /* Combinator copil direct: <li> direct sub <ul> */
17 ul > li { margin-bottom: 0.5rem; }
18
19 /* Combinator frate adiacent: <p> imediat dupa <h2> */
20 h2 + p { margin-top: 0; }
21
22 /* Pseudo-clase: stari ale elementului */
23 .cta:hover { background: red; }
24 .cta:focus-visible { outline: 2px solid blue; }
25 .cta:disabled { opacity: 0.5; }
26
27 /* Pseudo-elemente: parti generate */
28 p::first-letter { font-size: 2em; }
```

2.1 Specificitate – cum se rezolva conflictele

Cand mai multe reguli vizeaza acelasi element, browserul calculeaza o specificitate pentru fiecare si o aplica pe cea cu valoare maxima. Calculul foloseste un triplet (a, b, c):

- a – numarul de id-uri.
- b – numarul de clase, attribute, pseudo-clase.
- c – numarul de tag-uri si pseudo-elemente.

Comparatia se face lexicografic: (0,1,0) pierde fata de (1,0,0) indiferent de b si c. Stilurile inline au specificitate (1,0,0,0), peste orice selector. Cuvantul cheie `!important` suprascrie tot, dar este considerat un *antipattern* – indica faptul ca arhitectura CSS este compromisa.

✘ ASA NU – FOLOSIREA `!important` CA SOLUTIE

```
.btn { background: red !important; }
```

Problema. Cand alta regula are nevoie sa schimbe culoarea, va trebui si ea sa foloseasca `!important`, escaladand conflictul.

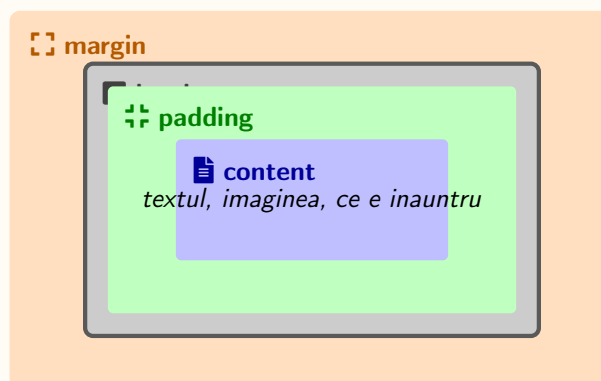
✔ ASA DA – SPECIFICITATE PROPORTIONALA

```
.btn { background: blue; } .btn.btn-danger { background: red; }
```

Solutie: clase modificador (`.btn-danger`) ridica specificitatea natural cu (0,2,0) fata de (0,1,0).

3. Modelul de cutie

Fiecare element redat de browser este o cutie compusa din patru straturi concentrice: *content*, *padding*, *border*, *margin*.



- **content** – zona ocupata de continutul efectiv (text, imagine).
- **padding** – spatiul dintre continut si chenar.
- **border** – chenarul.
- **margin** – spatiul exterior, intre cutia curenta si elementele adiacente.

3.1 Implicit vs border-box

Comportamentul implicit (`box-sizing: content-box`) include in `width` doar continutul. Adaugarea de `padding` sau `border` maresc latimea totala. Acest comportament face calculele predispuse la erori.

✖ ASA NU – CONTENT-BOX, LATIMI IMPREVIZIBILE

```
.card {
  width: 300px;
  padding: 20px;
  border: 1px solid #ccc;
  /* Latime totala: 300 + 20*2 + 1*2 = 342px (oversize) */
}
```

✔ ASA DA – BORDER-BOX GLOBAL, LATIMI PREDICTIBILE

```
*, *::before, *::after { box-sizing: border-box; }

.card {
  width: 300px;
  padding: 20px;
  border: 1px solid #ccc;
  /* Latime totala: 300px (padding si border absorbite in width)
   */
}
```

4. Unitati de masura

CSS suporta zeci de unitati. In practica, doar 5–6 acopera majoritatea cazurilor:

- **px** – pixel absolut. Pentru border-uri si dimensiuni de detaliu.
- **rem** – relativ la font-size-ul `<html>` (implicit 16px). Pentru font-size si spatiere uniforma.
- **em** – relativ la font-size-ul elementului parinte. Util pentru padding-uri scaland cu textul.
- **%** – relativ la dimensiunea parintelui pe acea axa.
- **vw / vh** – 1% din latimea / inaltimea viewport-ului.
- **ch** – latimea unui caracter „0”. Util pentru `max-width` pe paragrafe.

💡 PONT

Pentru font-size si spatiere generala, **rem** este alegerea recomandata. Cand utilizatorul mareste font-size-ul implicit din browser (Settings → Appearance), tot site-ul scaleaza proportional. **px** pe font-size ignora aceasta preferinta.

Listing 6: Compunere uzuala

```
1 :root {
2   font-size: 100%;           /* echivalent 16px in browserele
   standard */
3 }
4
5 body { font-size: 1rem; }    /* 16px */
6 h1   { font-size: 2.5rem; } /* 40px */
```

```

7 .container { max-width: 64rem; }      /* 1024px */
8
9 p { max-width: 65ch; }                /* lungime optima de citire */
10
11 .hero {
12   min-height: 80vh;                  /* 80% din inaltimea ferestrei */
13   padding: 2rem;
14 }

```

5. Variabile CSS (custom properties)

Variabilele CSS permit definirea valorilor o singura data si reutilizarea lor pe parcursul intregului document. Modificarea unei singure declaratii in `:root` se propaga automat in toate locurile in care variabila este folosita.

```

1 :root {
2   /* Paleta */
3   --primary: #0673BA;
4   --accent:  #FAA51E;
5   --bg:      #FFFCF6;
6   --text:    #1A1A1A;
7
8   /* Spatiere consistenta */
9   --space-1: 0.5rem;
10  --space-2: 1rem;
11  --space-3: 2rem;
12
13  /* Layout */
14  --max-width: 64rem;
15  --radius: 0.5rem;
16 }
17
18 body {
19   background: var(--bg);
20   color: var(--text);
21 }
22
23 h1 { color: var(--primary); }
24 .cta { background: var(--accent); }

```

Variabilele CSS sunt *scoped*: redefinirea intr-un selector descendent suprascrie valoarea pentru subarbore. Acest comportament permite teme contextuale fara duplicarea regulilor:

Listing 7: Tema schimbata local

```

1 .card-dark {
2   --bg:    #1F2937;
3   --text:  #F1F5F9;
4   /* toate copiii care folosesc var(--bg) si var(--text) preiau
      noile valori */

```

```

}

```

6. Proprietatea display – valori

`display` controleaza modul fundamental in care elementul se aseaza in pagina. Cele 6 valori pe care le intalnesti in 99% din cazuri:

Valoare	Efect
<code>block</code>	Element pe linie noua, ocupa toata latimea disponibila. Default pentru <code><div></code> , <code><p></code> , <code><h1>...<h6></code> , <code><section></code> .
<code>inline</code>	Curge in textul din jur, ignora <code>width/height</code> . Default pentru <code></code> , <code><a></code> , <code></code> , <code></code> .
<code>inline-block</code>	Curge in text ca inline, dar accepta <code>width/height</code> ca un block. Util pentru butoane in text.
<code>flex</code>	Activeaza modelul Flexbox (uni-dimensional) pe copiii elementului. Cel mai folosit pentru header-uri, navigatie, alinieri.
<code>grid</code>	Activeaza modelul CSS Grid (bi-dimensional) pe copii. Pentru galerii, dashboard-uri, layout-uri complexe.
<code>none</code>	Ascunde complet elementul (nu ocupa spatiu). Util pentru meniuri mobile inchise, modale ascunse.

⚠ ATENȚIE

`display: none` ascunde complet elementul (cititoarele de ecran il sar). Daca vrei doar sa-l faci invisibil dar accesibil, foloseste `visibility: hidden` sau `opacity: 0; pointer-events: none`.

7. Flexbox

Flexbox (*Flexible Box Layout*) este un model de layout uni-dimensional, util pentru distribuirea spatiului si alinierea continutului pe o axa (orizontala sau verticala).

Listing 8: Header cu logo si navigare aliniate

```

1 header {
2     display: flex;                                /* activeaza flexbox */
3     flex-direction: row;                          /* axa principala = orizontala
      (default) */
4     justify-content: space-between;               /* distribuire pe axa principala
      */
5     align-items: center;                          /* aliniere pe axa secundara */
6     flex-wrap: wrap;                             /* trecere pe randul urmator daca
      nu incapa */
7     gap: var(--space-2);                         /* spatiu intre copii */
8 }

```

7.1 Cele 6 proprietati pe care le folosesti

- `display: flex` – activeaza modelul (pe parinte).
- `flex-direction` – axa principala: `row` (default, orizontal), `column` (vertical), `row-reverse`, `column-reverse`.
- `justify-content` – distributie pe axa principala. Vezi tabelul de mai jos.
- `align-items` – aliniere pe axa secundara. Vezi tabelul de mai jos.
- `gap` – spatiu uniform intre copii (in unitati: `1rem`, `16px`). Inlocuieste `margin`.
- `flex-wrap` – `nowrap` (default, totul pe o linie), `wrap` (trece pe linia urmatoare).
- `flex: 1` (pe copil) – crește pentru a umple spatiul disponibil.

Valorile pentru `justify-content` (axa principala)

<code>flex-start</code> (default)	Toate copii lipiti la inceputul axei.
<code>center</code>	Grupate la mijlocul axei.
<code>flex-end</code>	Lipiti la finalul axei.
<code>space-between</code>	Primul/ultimul la margini, spatiu egal intre restul.
<code>space-around</code>	Spatiu egal in jurul fiecaruia (margine = jumătate).
<code>space-evenly</code>	Spatiu absolut egal peste tot (inclusiv la margini).

Valorile pentru `align-items` (axa secundara)

<code>stretch</code> (default)	Copii se intind pe toata latimea axei secundare.
<code>center</code>	Centrate pe axa secundara.
<code>flex-start</code>	Lipiti la inceputul axei (sus pentru row, stanga pentru column).
<code>flex-end</code>	Lipiti la final (jos / dreapta).
<code>baseline</code>	Aliniasi dupa linia de baza a textului (util pentru fonturi de marimi diferite).

💡 PONT

Trick uzual: `display: flex; justify-content: center; align-items: center;` centreaza orice element in containerul sau, atat orizontal cat si vertical, in 3 linii.

8. CSS Grid (introducere scurta)

Grid este sistemul de layout bidimensional al CSS-ului. Spre deosebire de flexbox (uni-dimensional), grid permite controlul simultan al randurilor si coloanelor.

Listing 9: Galerie cu coloane responsive automat

```
.gallery {
```

```

2   display: grid;
3   grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
4   gap: var(--space-2);
5 }

```

Mecanism: `auto-fit` creeaza cate coloane incap; `minmax(240px, 1fr)` cere o latime minima de 240px si o expansiune egala (`1fr`) cand ramane spatiu. Rezultatul: galerie responsive fara media queries.

9. Mobile-first

Strategia *mobile-first* consta in scrierea regulilor pentru ecrane mici ca valori implicite, urmata de extinderea acestora pentru ecrane mai mari prin *media queries*:

```

1  /* Stiluri implicite, pentru mobil */
2  .nav { display: none; }
3  .nav-toggle { display: flex; }
4
5  /* Adaugiri pentru ecrane >= 768px */
6  @media (min-width: 768px) {
7    .nav { display: flex; }
8    .nav-toggle { display: none; }
9  }

```

✗ ASA NU – DESKTOP-FIRST CU MAX-WIDTH

```

.nav { display: flex; }
@media (max-width: 767px) {
  .nav { display: none; }
  .nav-toggle { display: flex; }
}

```

Problema. Mobilul descarca si proceseaza CSS conceput pentru desktop, apoi il anuleaza. Pierdere de performanta pe contextul cu cele mai limitate resurse.

✓ ASA DA – MOBILE-FIRST CU MIN-WIDTH

```

.nav { display: none; }
.nav-toggle { display: flex; }
@media (min-width: 768px) {
  .nav { display: flex; }
  .nav-toggle { display: none; }
}

```

Beneficiu. Mobilul (>60% din trafic) primeste varianta optimizata implicit; desktopul adauga incremental.

10. Suport pentru tema sistem (light / dark)

Funcția `light-dark()` și proprietatea `color-scheme` permit afisarea conditionata a culorilor în funcție de preferința sistemului de operare al utilizatorului:

```
1 html { color-scheme: light dark; }
2
3 :root {
4   --bg:    light-dark(#FFFCF6, #0F172A);
5   --text:  light-dark(#1A1A1A, #F1F5F9);
6 }
```

Browserul selectează prima sau a doua valoare în funcție de tema activă. Suportul nativ în browserele moderne este disponibil începând cu 2024 (Chrome 123+, Firefox 120+, Safari 17.5+).

11. Poziționarea elementelor

Proprietatea `position` controlează modul în care un element se așază în fluxul paginii. Cinci valori, cu reper de când le folosești:

Valoare	Comportament	Când folosește
<code>static</code> (default)	Respecta fluxul natural; <code>top/left</code> ignorate.	implicit, nu îl setezi
<code>relative</code>	Stay-in-place până îl muți cu <code>top/left</code> ; păstrează spațiul.	ancora pentru <code>absolute</code>
<code>absolute</code>	Scos din flux; poziționat față de primul părinte <code>relative</code> .	tooltip-uri, badge-uri
<code>fixed</code>	Fixat pe viewport; rămâne la scroll.	modal, buton „scroll-to-top”
<code>sticky</code>	<code>relative</code> până ajunge la o margine, apoi <code>fixed</code> .	header sticky, sidebar

⚠ ATENȚIE

`absolute` are nevoie de un părinte `relative` (sau cade până la viewport). Dacă elementul „sare în lume”, verifică dacă containerul are `position: relative`.

Listing 10: Header sticky modern

```
1 header {
2   position: sticky;
3   top: 0;
4   background: var(--bg);
5   z-index: 10;
6 }
```


12. Cheatsheet – alte proprietăți cu enum-uri

Cele patru cele mai uzuale după cele de mai sus. Le întâlnești zilnic.

Proprietate	Valori	Când folosește
<code>box-sizing</code>	<code>content-box</code> <code>border-box</code>	(default), <i>Mereu <code>border-box</code> – aplicat global.</i>
<code>overflow</code>	<code>visible</code> (default), <code>hidden</code> , <code>scroll</code> , <code>auto</code>	<code>hidden</code> pentru a tunde, <code>auto</code> pentru scroll doar la nevoie.
<code>text-align</code>	<code>left</code> (default), <code>center</code> , <code>right</code> , <code>justify</code>	Aliniere text în container.
<code>cursor</code>	<code>auto</code> , <code>pointer</code> , <code>default</code> , <code>text</code> , <code>grab</code> , <code>not-allowed</code> , <code>wait</code>	<code>pointer</code> pe elemente clicabile non-link.
<code>font-weight</code>	<code>normal</code> (400), <code>bold</code> (700), 100–900 în pași de 100	500 pentru <i>medium</i> , 600 pentru <i>semi-bold</i> .
<code>text-transform</code>	<code>none</code> (default), <code>uppercase</code> , <code>lowercase</code> , <code>capitalize</code>	În CSS, nu în HTML – păstrezi semantică.
<code>visibility</code>	<code>visible</code> (default), <code>hidden</code>	Ascunde dar păstrează spațiul; <code>display: none</code> îl scoate complet.
<code>pointer-events</code>	<code>auto</code> (default), <code>none</code>	<code>none</code> dezactivează click și hover (overlay-uri inactive).

🔗 PONT

Regula generală: valori unitare ca `cursor: pointer`, `text-align: center`, `overflow: hidden` arată exact ca în tooltip-ul autocomplete al VS Code. `Ctrl+Space` după numele proprietății afișează lista completă.

13. Animații CSS

Animațiile îmbunătățesc percepția calității produsului și orientează atenția utilizatorului. CSS suporta nativ două mecanisme: *transitions* pentru schimbări între stări, și *keyframes* pentru animații arbitrare.

13.1 Transitions – schimbări între stări

`transition` interpolează fluent o proprietate când valoarea sa se schimbă (de exemplu, la `:hover`).

Listing 11: Buton cu hover smooth

```

1 .cta {
2   background: var(--primary);
3   color: white;
4   padding: var(--space-1) var(--space-3);
5   border-radius: var(--radius);
6   transition: background 0.2s ease, transform 0.2s ease;
7 }
```

```

8
9 .cta:hover {
10     background: var(--accent);
11     transform: translateY(-2px);
12 }

```

Sintaxa: `transition: <proprietate> <durata> <timing-function> <delay>.`

Timing function – cum se simte miscarea

Durata spune *cat* dureaza animatia; *timing function* spune *cum* se distribuie miscarea in timp. Valorile predefinite sunt suficiente pentru 90% din cazuri:

- `linear` – viteza constanta. Util pentru spinner-e si bare de progres.
- `ease` (default) – accelereaza la inceput, decelereaza la final.
- `ease-in` – porneste lent, se incheie rapid. Folosit pentru iesiri (*exit*).
- `ease-out` – porneste rapid, se incheie lent. Folosit pentru intrari (*enter*); senzatie de aterizare lina.
- `ease-in-out` – simetric. Folosit pentru tranzitii reversibile (*toggle*).

💡 REGULA UX RAPIDA

Pentru micro-interactiuni (hover, click): `ease-out 150ms-250ms`. Sub 150ms se simte abrupt; peste 300ms se simte lent.

Cubic-bezier – curba personalizata

Cele 5 valori predefinite sunt scurtaturi pentru curbe Bezier de gradul 3. Cand vrei o curba personalizata (ex. *bounce*, *overshoot*), o definești prin 4 numere care reprezinta cele doua puncte de control:

Listing 12: Easing personalizat cu overshoot subtil

```

1 .cta {
2     transition: transform 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
3 }
4 .cta:hover { transform: scale(1.05); }

```

💡 PONT

Tool recomandat: <https://cubic-bezier.com> – editor vizual cu preview. Copiezi cele 4 valori si le lipesti in CSS.

13.2 Transform – scalare, rotație, translație

`transform` aplica o transformare geometrica fara a afecta layout-ul (alte elemente nu se mută).

Operatiile uzuale:

- `translate(x, y)` / `translateX` / `translateY` – mutare pe axe (px, %, rem). Mai performant decat `top/left`.

- `scale(n)` – marire/micsorare proportionala. `scaleX/scaleY` pe o singura axa.
- `rotate(deg)` – rotire in plan; pozitiv = sensul acelor de ceas.
- `skew(deg)` – forfecare (rar folosit, util pentru efecte oblique).
- Combinare *ordonata*: `transform: translateY(-2px) scale(1.05) rotate(2deg)` – ordinea conteaza, se aplica de la dreapta la stanga.

Transform-origin – punctul de pivot

Implicit, transformarile se aplica fata de centrul elementului. `transform-origin` schimba acest pivot:

Listing 13: Card care se „deschide” din coltul stanga-sus

```
1 .card {
2   transform-origin: top left;
3   transition: transform 0.3s ease-out;
4 }
5 .card:hover { transform: scale(1.05); }
```

Valori uzuale: `center` (default), `top`, `bottom`, `left`, `right`, combinatii (`top left`, `50% 0%`), sau coordonate explicite in `px/%`.

🔗 PERFORMANCE

Browserul accelereaza pe GPU doar `transform` si `opacity`. Animația altor proprietăți (ex. `width`, `margin`, `top`) declanșează reflow și scade FPS. Regula: pentru mișcare, folosește `transform: translate(...)`, nu `top/left`.

13.3 Keyframes – animatii arbitrare

Pentru animații care nu sunt declanșate de o stare (loading spinner, fade-in, parallax), `@keyframes` definește o secvență de stari:

Listing 14: Spinner infinit

```
1 @keyframes rotate {
2   from { transform: rotate(0deg); }
3   to   { transform: rotate(360deg); }
4 }
5
6 .spinner {
7   width: 32px;
8   height: 32px;
9   border: 4px solid #E5E7EB;
10  border-top-color: var(--primary);
11  border-radius: 50%;
12  animation: rotate 1s linear infinite;
13 }
```

Listing 15: Fade-in pentru titlu hero

```

1 @keyframes fadeInUp {
2   from { opacity: 0; transform: translateY(20px); }
3   to   { opacity: 1; transform: translateY(0); }
4 }
5
6 .hero h1 {
7   animation: fadeInUp 0.6s ease-out;
8 }

```

Sintaxa shorthand pentru animation: `animation: <nume> <durata> <timing> <delay> <iteratii> <direction> <fill-mode>.`

Keyframes multi-step

`@keyframes` accepta procente intermediare, nu doar `from` si `to`. Astfel poți descrie traiectorii complexe într-o singura animație:

Listing 16: Buton care „pulsează” atragator

```

1 @keyframes pulse {
2   0%   { transform: scale(1);   box-shadow: 0 0 0 0 rgba(250, 165,
3         30, 0.7); }
4   50%  { transform: scale(1.05); box-shadow: 0 0 0 12px rgba(250,
5         165, 30, 0); }
6   100% { transform: scale(1);   box-shadow: 0 0 0 0 rgba(250, 165,
7         30, 0); }
8 }
9
10 .cta-featured { animation: pulse 2s ease-in-out infinite; }

```

Animation-fill-mode – ce ramane dupa animație

Implicit, după ce animația se termină, elementul revine la starea CSS originală. `animation-fill-mode` controlează acest comportament:

- `none` (default) – elementul revine la stilul original.
- `forwards` – păstrează valorile din ultimul keyframe. **Cel mai folosit** pentru fade-in / slide-in (vrei să rămână la `opacity: 1`).
- `backwards` – aplică primul keyframe înainte de start (în timpul `animation-delay`).
- `both` – combinație `forwards` + `backwards`.

Listing 17: Fade-in care rămâne vizibil

```

1 .hero h1 {
2   opacity: 0;
3   animation: fadeInUp 0.6s ease-out forwards;
4 }

```

Animation-play-state – pauză și reluare

`animation-play-state` permite punerea pe pauza a unei animații – util pentru carusele care se opresc la hover:

```
.carousel { animation: scroll 30s linear infinite; }
.carousel:hover { animation-play-state: paused; }
```

13.4 Pattern complet: card cu hover-lift

Combinatia `transition` + `transform` + `box-shadow` produce efectul de „ridicare” pe care îl vezi pe orice landing page modern:

Listing 18: Hover-lift, pattern de bază pentru carduri

```
1 .card {
2   background: var(--bg);
3   padding: var(--space-3);
4   border-radius: var(--radius);
5   box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
6   transition: transform 0.25s ease-out, box-shadow 0.25s ease-out;
7   cursor: pointer;
8 }
9
10 .card:hover {
11   transform: translateY(-4px);
12   box-shadow: 0 12px 24px rgba(0, 0, 0, 0.12);
13 }
```

Detalii care fac diferența:

- `ease-out` pe enter – senzație de aterizare lină.
- Umbra crește și se „răsfră” (offset Y mai mare, blur mai mare) – simulează distanța față de fundal.
- Tranziție pe ambele proprietăți simultan, nu doar `transform`.
- Cursor `pointer` – afișează intenția ca elementul e clicabil.

13.5 Accesibilitate: `prefers-reduced-motion`

Unii utilizatori (afecțiuni vestibulare, dependente de atenție) au setat în OS preferința pentru animații reduse. CSS detectează această cerere:

```
1 @media (prefers-reduced-motion: reduce) {
2   *, *::before, *::after {
3     animation-duration: 0.01ms !important;
4     transition-duration: 0.01ms !important;
5   }
6 }
```

Conformitatea este o cerință WCAG 2.1 (criteriul 2.3.3) și o practică recomandată pentru orice

site cu animatii.

BONUS • AVANSAT

Acest box e **optional**. Dacă ești la început, treci direct la *Verificare cunoștințe*. Secțiunile de mai jos sunt pentru când vrei să ridici stacheta.

Transformări 3D – flip-card

CSS suporta nativ transformări în spațiu, nu doar în plan. Trei piese:

- `perspective` (pe părinți) – „distanța” față de plan. Valori uzuale 600–1500px.
- `transform-style: preserve-3d` (pe părinți) – permite copiilor să existe în spațiu 3D.
- `backface-visibility: hidden` (pe fete) – ascunde fața din spate.

Listing 19: Card care se întoarce la hover (flip-card)

```
1 .flip-card { perspective: 1000px; }
2 .flip-card-inner {
3   position: relative;
4   width: 100%; height: 100%;
5   transition: transform 0.6s;
6   transform-style: preserve-3d;
7 }
8 .flip-card:hover .flip-card-inner { transform: rotateY(180deg); }
9 .flip-card-front, .flip-card-back {
10  position: absolute; inset: 0;
11  backface-visibility: hidden;
12 }
13 .flip-card-back { transform: rotateY(180deg); }
```

Animații legate de scroll – animation-timeline

CSS modern (Chrome 115+, 2023) permite legarea unei animații de poziția scroll-ului, fără JavaScript:

Listing 20: Bara de progres care urmărește scroll-ul

```
1 @keyframes growProgress {
2   from { transform: scaleX(0); }
3   to   { transform: scaleX(1); }
4 }
5
6 .progress-bar {
7   position: fixed; top: 0; left: 0;
8   height: 4px; width: 100%;
9   background: var(--accent);
10  transform-origin: left;
11  animation: growProgress linear;
12  animation-timeline: scroll(root); /* legat de scroll-ul
13                                     paginii */
14 }
```

🔗 continuare bonus

view-timeline este varianta complementară: animația avansează pe măsura ce un element intră în viewport. Util pentru fade-in la scroll, dar fără cost JavaScript.

📌 NOTĂ

Suportul pentru **animation-timeline** este parțial în 2026: Chrome/Edge da, Firefox în spațiile unui flag, Safari în pregătire. Folosește cu fallback (animația pur și simplu nu rulează în browserele vechi – progressive enhancement OK).

🔗 filter și backdrop-filter – efecte de imagine

filter aplică un efect vizual pe element. **backdrop-filter** aplică același efect pe ce este *în spate*: așa se face glassmorphism-ul (efect „sticlă mată”).

Listing 21: Header transparent cu blur pe conținutul dedesubt

```
1 header {
2   background: rgba(255, 252, 246, 0.7);
3   backdrop-filter: blur(12px) saturate(180%);
4   -webkit-backdrop-filter: blur(12px) saturate(180%); /*
5     Safari */
6 }
```

Funcții utile: `blur(8px)`, `brightness(1.2)`, `contrast(1.1)`, `grayscale(80%)`, `hue-rotate(15deg)`, `drop-shadow(0 4px 8px rgba(0,0,0,0.2))`. Pot fi combinate prin spațiere.

✂ clip-path – forme custom și reveal effects

clip-path „decupează” elementul după o formă. Util pentru hero-uri oblice, butoane în forma de chevron, animații de tip „revelare”:

Listing 22: Imagine cu margine diagonală

```
1 .hero-image {
2   clip-path: polygon(0 0, 100% 0, 100% 85%, 0 100%);
3 }
```

Listing 23: Reveal animat: pagina se „deschide” la load

```
1 @keyframes reveal {
2   from { clip-path: inset(0 100% 0 0); }
3   to   { clip-path: inset(0 0 0 0); }
4 }
5 .hero h1 { animation: reveal 0.8s ease-out forwards; }
```

💡 PONT

Tool vizual: <https://bennettfeely.com/clippy/> – editor SVG-like care generează valoarea **clip-path** pentru orice formă desenată.

📦 Container queries – „media queries” pentru componente

🔗 continuare bonus

Problema cu `@media`: depinde de viewport, nu de containerul real al componente. Un card de 300px se comporta diferit intr-o coloana ingusta vs pe ecran lat, dar `@media` nu vede asta. `@container` (CSS 2023, suport stabil) rezolva exact asta:

Listing 24: Card responsive in functie de container, nu de viewport

```
1 .card-wrapper { container-type: inline-size; }
2
3 .card { display: block; }
4
5 @container (min-width: 400px) {
6   .card { display: flex; gap: var(--space-2); }
7 }
```

Același card devine „horizontal” cand are spatiu, „vertical” cand nu – fara sa-i pese de marimea ecranului. Standardul modern pentru design system-uri.

🔗 Unde mergi de aici

- Animatable properties: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_animated_properties
- Scroll-driven animations: <https://developer.chrome.com/docs/css-ui/scroll-driven-animations>
- Container queries: <https://web.dev/learn/design/container-queries>

Verificare cunoștințe

☰ PROVOCARE

1. Care este specificitatea pentru selectorul `.btn.primary`? Comparativ, ce specificitate are `button#submit`?
2. De ce strategia mobile-first este preferabila pentru contextul actual al traficului web?
3. Spot the issue: `.btn { background: red !important; }` repetat in trei locuri.
4. Care este diferența funcționala între `rem` și `px` pentru `font-size`?
5. Ce face `box-sizing: border-box` față de comportamentul implicit?
6. Pentru un hover smooth pe un buton, ce duration și timing-function alegi? Argumentează.
7. De ce `transform: translateY(-2px)` pe hover este preferabil față de `top: -2px`?
8. Cand folosești `animation-fill-mode: forwards`? Da un exemplu concret.

Recapitulare

✅ FAZA 2 – CSS

- Stiluri externe în `<link rel="stylesheet">`; nu inline.
- Specificitate: (`id`, `clasa`, `tag`); `!important` este *antipattern*.
- `box-sizing: border-box` aplicat global pentru calcul predictibil.

- Unitati: `rem` pentru fonturi/spatiere; `px` pentru border; `%/vw/vh` pentru layout responsiv.
- Variabilele CSS in `:root`; redefinire locala pentru teme contextuale.
- Flexbox: 6 proprietati cheie (`display`, `flex-direction`, `justify-content`, `align-items`, `gap`, `flex: 1`).
- Mobile-first cu `@media (min-width: ...)`, nu desktop-first cu `max-width`.
- Pozitionare: `static` (default), `relative`, `absolute`, `fixed`, `sticky`.
- Animații: `transition` pentru schimbari de stare (hover/focus), `@keyframes` pentru animatii arbitrare.
- Timing function: `ease-out` pentru intrari, `ease-in` pentru iesiri, `cubic-bezier()` pentru curbe custom.
- Performance: anima doar `transform` și `opacity` (accelerate GPU). Evita `top/left/width` - declanseaza reflow.
- Pattern hover-lift: `transform: translateY(-4px) + box-shadow` cu `transition` pe ambele.
- `animation-fill-mode: forwards` pastreaza ultimul keyframe (esential pentru fade-in care ramane vizibil).
- Accesibilitate: respecta `prefers-reduced-motion`.
- 🐼 Bonus: 3D transforms, scroll-driven animations, `filter`, `clip-path`, container queries.



🔧 APLICAȚIE PRACTICĂ

Platforma: Flexbox Froggy (<https://flexboxfroggy.com>).

Activitatea. Aplicatia este un set de 24 de provocari care necesita pozitionarea unor obiecte pe o grila prin folosirea proprietatilor `justify-content`, `align-items`, `flex-direction`, `order`, `flex-wrap`.

Obiectiv: primele 6 niveluri (acopera `justify-content` si `align-items`, cele mai folosite proprietati flexbox in practica).

Durata estimata: 5 minute.

🎓 Resurse pentru aprofundare

- Mozilla Developer Network. *CSS reference*. <https://developer.mozilla.org/en-US/docs/Web/CSS>
- Google. *Learn CSS*. <https://web.dev/learn/css>
- Coyier C. *A Complete Guide to Flexbox*. CSS-Tricks. <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>
- Coyier C. *A Complete Guide to Grid*. CSS-Tricks. <https://css-tricks.com/snippets/css/complete-guide-grid/>
- The Odin Project. *CSS Foundations*. <https://www.theodinproject.com/paths/foundations/courses/foundations#css-foundations>
- Aplicatii similare: <https://flukeout.github.io> (selectori), <https://cssgridgarden.com>

(grid), <https://cssbattle.dev> (speedrun).

→ **Urmeaza:** Capitolul 3 (JavaScript) adauga interactivitate. Pagina stilizata in Faza 2 va raspunde la click-uri, va schimba dark mode, va procesa formulare – fara reincarcare.

Faza 3 – JavaScript

Variabile. Functii. Pattern-ul SELECT-LISTEN-CHANGE. Manipulare DOM.

JavaScript este limbajul de programare standardizat ECMAScript (ECMA-262), interpretat nativ de toate browserele moderne. Spre deosebire de HTML (structura) și CSS (aspect), JavaScript adauga **comportament**: raspunde la actiunile utilizatorului, modifica continutul paginii dinamic, comunica cu servere prin retea.

Capitolul prezinta sintaxa de baza, modelul de programare orientat-eveniment specific paginilor web statice, și interactiunea cu arborele DOM (*Document Object Model*).

🕒 CE ÎNVEȚI

Dupa capitolul asta vei ști:

- Declara variabile cu **const** și **let**, recunoscand cand fiecare se aplica.
- Aplica pattern-ul **SELECT** → **LISTEN** → **CHANGE** pentru orice interacțiune.
- Manipuleaza DOM-ul cu **querySelector**, **classList**, **addEventListener**.
- Intercepteaza submit-ul unui formular și extrage valorile cu **FormData**.
- Folosește **IntersectionObserver** pentru animații și lazy loading legate de scroll.
- Diagnostheaza erorile JavaScript folosind console-ul browserului.
- 🐞 *Bonus self-study*: debounce/throttle, event delegation, **requestAnimationFrame**, **fetch** cu **async/await**, metode pe array (**map/filter**), **localStorage**.

❓ DE CE

HTML și CSS construiesc o pagina statica. JavaScript transforma pagina intr-o aplicatie: poate reacționa la click-uri, poate schimba conținutul fara reincarcare, poate trimite cereri catre servere. Pattern-ul SELECT-LISTEN-CHANGE prezentat aici acopera 80% din scenariile de interactivitate și este aceeași abstracție pe care React, Vue și Svelte o ambala in API-uri declarative.

📖 PRE-RECHIZITE

Capitolul presupune înțelegerea structurii HTML (tag-uri, attribute) și a selectorilor CSS (clase, id-uri). Faze 1 și 2.

1. Conectarea fisierului JavaScript la HTML

Includerea unui fisier JavaScript se realizeaza prin elementul `<script>`, plasat in `<head>` cu atributul **defer**:

```
<script src="script.js" defer></script>
```

1.1 defer vs async vs nimic

- **Fara atribut**. Browserul opreste parsarea HTML-ului, descarca si ruleaza scriptul, apoi continua. Blocheaza randarea – antipattern.

- **async** – descarcare in paralel cu parsarea, dar executie imediat ce scriptul este disponibil (poate intrerupe parsarea). Util doar pentru scripturi independente (analytics).
- **defer** – descarcare in paralel, executie **dupa** parsarea HTML-ului. Garantat executat in ordinea declararii. Optiunea recomandata pentru codul propriu.

⚠ ATENȚIE

Fara **defer**, scriptul ruleaza inainte de parsarea `<body>`: `document.querySelector('.cta')` returneaza `null`. Cu **defer**, intregul DOM exista cand ruleaza scriptul.

2. Declararea variabilelor

ECMAScript 2015 introduce doua cuvinte cheie pentru declararea variabilelor: **const** (valoare imuabila) si **let** (valoare mutabila). Cuvantul cheie **var** este considerat depasit din cauza regulilor de scope.

Listing 25: Reguli de declarare

```
1 const PI = 3.14;           // valoare imutabila
2 let counter = 0;          // valoare mutabila
3
4 counter = counter + 1;     // permis
5 // PI = 3.15;             // TypeError la rulare
```

2.1 Diferenta dintre var si let / const

var are *function scope*: o variabila declarata in interiorul unui **if** sau **for** este accesibila in tot blocul functiei. **let** si **const** au *block scope*: vizibile doar in interiorul `{...}` in care au fost declarate.

✘ ASA NU – VAR, SCOPE CONFUZ

```
function exemplu() {
  if (true) {
    var x = 10;
  }
  console.log(x); // 10 -- scapa din if!
}
```

✔ ASA DA – LET, SCOPE PREVIZIBIL

```
function exemplu() {
  if (true) {
    let x = 10;
  }
  console.log(x); // ReferenceError -- nu exista in afara
                  if-ului
}
```

Conventia practica: **const** este valoarea implicita; **let** se foloseste doar cand reassignarea este

necesara; `var` se evita complet in cod nou.

3. Tipuri primitive si compuse

```

1 const nume      = "Voluntar";           // string
2 const varsta    = 21;                   // number (intregi si
   zecimale)
3 const eStudent  = true;                 // boolean
4 const limbaje   = ["HTML", "CSS"];      // array (lista ordonata)
5 const persoana  = {                     // object (pereche
   cheie-valoare)
6   nume: "Voluntar",
7   varsta: 21,
8 };
9
10 console.log(persoana.nume);             // "Voluntar"
11 console.log(limbaje[0]);                 // "HTML"
12 console.log(typeof varsta);              // "number"

```

JavaScript este un limbaj cu *tipare dinamica*: tipul unei variabile este determinat de valoarea atribuita si poate fi modificat in timpul executiei.

3.1 Verificarea egalitatii: `===` vs `==`

Operatorul `==` efectueaza *coercitie de tip*: converteste operanzii inainte de comparatie. Rezultatul este des contraintuitiv. Operatorul `===` compara strict valoare si tip.

✗ ASA NU – `==`, COERCITIE SURPRINZATOARE

```

0 == "0"           // true
0 == ""            // true
"" == false        // true
null == undefined  // true
[] == false        // true

```

✓ ASA DA – `===` MEREU

```

0 === "0"          // false (number vs string)
0 === ""           // false
null === undefined // false

```

4. Functii: declaratie, expresie, arrow

JavaScript ofera trei sintaxe pentru definirea unei functii. Cele trei comportamente difera la nivel de `this` si hoisting, dar pentru cazul uzual (handler de eveniment) sunt interschimbabile.

Listing 26: Trei moduri de a scrie aceeasi functie

```

1 // 1. Declaratie de functie (function declaration)

```

```

2 function saluta(ume) {
3   return `Salut, ${ume}!`;
4 }
5
6 // 2. Expresie de functie (function expression)
7 const saluta2 = function(ume) {
8   return `Salut, ${ume}!`;
9 };
10
11 // 3. Arrow function (sintaxa concisa)
12 const saluta3 = (ume) => `Salut, ${ume}!`;
13
14 // Apel: identic in toate cazurile
15 console.log(saluta("Voluntar"));

```

Conventia practica: arrow functions pentru handlers si callbacks; declaratii pentru functii cu nume reutilizate, definite la nivelul fisierului.

5. Pattern-ul SELECT → LISTEN → CHANGE

Majoritatea interactiunilor pe paginile statice respecta o schema in trei pasi: selectarea elementului DOM, atasarea unui ascultator de eveniment, modificarea elementului in raspuns la eveniment.

Listing 27: Toggle pentru tema dark/light

```

1 // 1. SELECT
2 const btn = document.querySelector('.toggle-theme');
3
4 // 2. LISTEN
5 btn.addEventListener('click', () => {
6
7   // 3. CHANGE
8   document.body.classList.toggle('dark');
9
10 });

```

🔗 PONT

Pattern-ul descris reprezinta abstractizarea fundamentala pe care framework-urile moderne (React, Vue, Svelte, Angular) o ascund in spatele propriilor API-uri declarative. Intelegerea variantei *vanilla* faciliteaza tranzitia ulterioara catre framework-uri.

6. Manipularea DOM-ului

API-ul `document` expune metode pentru selectia si modificarea nodurilor:

Listing 28: Operatii uzuale

```

1 // SELECT -- un singur element
2 const el = document.querySelector('.cta'); // primul .cta

```

```

3  const id = document.getElementById('contact');           // alternativa
   pt id
4
5  // SELECT -- mai multe elemente
6  const linkuri = document.querySelectorAll('nav a');
7  linkuri.forEach((link) => {
8    link.addEventListener('click', () => console.log('click'));
9  });
10
11 // CHANGE -- continut
12 el.textContent = 'Salut!';                                // text simplu (sigur)
13 el.innerHTML = '<strong>Salut!</strong>';                  // HTML (atentie XSS)
14
15 // CHANGE -- attribute
16 el.setAttribute('aria-expanded', 'true');
17 el.classList.add('active');
18 el.classList.remove('hidden');
19 el.classList.toggle('open');
20
21 // CHANGE -- creare element nou
22 const nou = document.createElement('li');
23 nou.textContent = 'Element nou';
24 document.querySelector('ul').appendChild(nou);

```

✖ ASA NU – QUERYSELECTOR LA FIECARE APEL

```

btn.addEventListener('click', () => {
  document.querySelector('.modal').classList.add('open');
  document.querySelector('.modal').focus();
  document.querySelector('.modal').scrollIntoView();
});

```

Problema. Trei traversari ale DOM-ului pentru aceeasi referinta.

✔ ASA DA – CACHE LOCAL

```

const modal = document.querySelector('.modal');

btn.addEventListener('click', () => {
  modal.classList.add('open');
  modal.focus();
  modal.scrollIntoView();
});

```

7. Procesarea formulelor

Submit-ul nativ al unui formular trimite o cerere HTTP catre URL-ul declarat in `action` si reincarca pagina. In majoritatea cazurilor, fluxul dorit este interceptarea evenimentului si manipularea da-

telor in JavaScript:

Listing 29: Captura datelor de formular fara reincarcarea paginii

```

1  const form = document.querySelector('#contact-form');
2
3  form.addEventListener('submit', (e) => {
4    e.preventDefault();                                // blocheaza
      reincarcarea
5
6    const data = Object.fromEntries(new FormData(form)); // {nume,
      email, mesaj}
7    console.log('Date primite:', data);
8
9    form.reset();                                    // goleste
      campurile
10   alert('Multumesc, ${data.nume}!');
11 });

```

Fluxul: `e.preventDefault()` suspenda comportamentul implicit; `new FormData(form)` colecteaza valorile dupa atributul `name`; `Object.fromEntries(...)` le converteste intr-un obiect uzual; `form.reset()` reseteaza valorile in interfata.

💡 PONT

Pentru transmiterea efectiva a datelor catre un endpoint extern (fara backend propriu), serviciul [Formspree](#) ofera un endpoint gratuit. Configurarea consta intr-o singura modificare a atributului `action`.

8. Debugging si console

Console-ul browserului (F12 → tab *Console*) este principalul instrument de diagnostic. Erorile JavaScript apar automat aici cu mesaj, fisier si linie.

Listing 30: Trei moduri de a inspecta valorile

```

1  // Cel mai uzual: log valoare cu eticheta
2  console.log('Date primite:', data);
3
4  // Tabel pentru obiecte / array-uri (formatare imbunatatita)
5  console.table(linkuri);
6
7  // Pauza in executie -- DevTools deschis
8  debugger;

```

Sfaturi practice. Inainte de a cere ajutor, citeste mesajul de eroare din console. Numele functiei si numarul liniei sunt aproape intotdeauna corecte. `console.log(typeof x)` verifica tipul cand comportamentul este surprinzator.

9. IntersectionObserver – animații la scroll

Cea mai uzuala cerere „fa o animație cand sectiunea devine vizibila” (fade-in pe scroll, reveal pe carduri, lazy loading pentru imagini) NU se rezolva cu `window.addEventListener('scroll', ...)`. Browserul ofera un API dedicat, mult mai eficient: `IntersectionObserver`.

Idea: in loc sa intrebi de 60 de ori pe secunda „e vizibila sectiunea?”, tu declari „anunța-ma cand starea de vizibilitate se schimba”. Browserul face restul, fara cost de performance.

Listing 31: Fade-in pe scroll: pattern canonic in 10 linii

```
1 const observer = new IntersectionObserver((entries) => {
2   entries.forEach((entry) => {
3     if (entry.isIntersecting) {
4       entry.target.classList.add('visible');
5       observer.unobserve(entry.target); // o singura data, nu repeti
6     }
7   });
8 }, { threshold: 0.15 }); // declanseaza cand 15% e in viewport
9
10 document.querySelectorAll('.reveal').forEach((el) =>
    observer.observe(el));
```

Pe CSS, partea sora:

```
.reveal { opacity: 0; transform: translateY(20px); transition: all
    0.6s ease-out; }
.reveal.visible { opacity: 1; transform: translateY(0); }
```

Cum se citește API-ul:

- Constructorul: `new IntersectionObserver(callback, optiuni)`.
- `callback(entries)` – se apeleaza cand starea de vizibilitate se schimba.
- `entry.isIntersecting` – `true` cand elementul intra in viewport.
- `entry.target` – elementul DOM (același pe care l-ai dat la `observe`).
- `threshold` – procent din element vizibil pentru a declanșa (`0` = orice pixel, `1` = complet vizibil, `0.15` = 15%).
- `rootMargin` – extinde/contracta viewport-ul (ex. "`0px 0px -100px 0px`" declanșează cu 100px înainte sa intre real in cadru).
- `observer.unobserve(el)` – oprește monitorizarea (perfect dupa prima activare).

✖ ASA NU – SCROLL HANDLER CU GETBOUNDCIENTRECT

```
window.addEventListener('scroll', () => {
  document.querySelectorAll('.reveal').forEach((el) => {
    if (el.getBoundingClientRect().top < window.innerHeight) {
      el.classList.add('visible');
    }
  });
});
```

```

    }
  });
});

```

Problema. Ruleaza la fiecare pixel de scroll (zeci – sute de ori pe secunda), forteaza recalculare layout (*layout thrashing*), scade FPS pe mobile.

✔ ASA DA – INTERSECTIONOBSERVER

Browserul observa eficient (compus pe thread separat). Callback-ul ruleaza doar cand starea se schimba. Zero impact pe performance.

💡 APLICAȚII DIRECTE

- Lazy loading custom pentru conținut dinamic (cand nu poti folosi `loading="lazy"`).
- Active state pentru navigare *scrollspy* (evidențiază link-ul activ in nav).
- Infinite scroll – cand userul ajunge aproape de jos, incarci urmatoarea pagina.
- Analytics „a vazut secțiunea X” (tracking eveniment doar cand secțiunea devine vizibila).

10. Tratarea erorilor (try/catch)

Codul care poate genera erori la rulare (parsare JSON, apeluri `fetch`, acces la API-uri externe) trebuie incadrat intr-un bloc `try/catch` pentru ca o eroare sa nu opreasca executia restului paginii:

Listing 32: Parsare JSON cu fallback

```

1 const raw = localStorage.getItem('preferinte');
2
3 try {
4   const config = JSON.parse(raw);
5   console.log('Preferinte incarcate:', config);
6 } catch (err) {
7   console.warn('JSON invalid in localStorage; folosesc valori
   implicite');
8   // continuare cu valori default
9 }

```

Cand eroarea este previzibila si recuperabila (utilizatorul poate continua), `try/catch` este abordarea corecta. Pentru erori neprevazute, evenimentul global `window.addEventListener('error', ...)` permite logarea catre un serviciu de monitoring.

11. Erori frecvente

- `Cannot read property '...' of null`. Cauza: `querySelector` a returnat `null` si codul a incercat sa acceseze o proprietate. Diagnostic: `console.log(e1)` inainte de a folosi.
- `addEventListener is not a function`. Aceeasi cauza: rezultatul `querySelector` este `null`.
- **Hoisting**. Variabilele `let` si `const` declarate intr-un bloc nu sunt accesibile inainte de declaratie. Accesul anticipat genereaza *ReferenceError: Cannot access before initialization*.

- **Comparatie cu `undefined`.** Foloseste `x === undefined`, nu `!x` – ultimul prinde si `0`, `""`, `false`, `null`.

🔧 BONUS • AVANSAT

Acest box e optional. Daca esti la inceput, treci direct la *Verificare cunoștințe*. Mai jos sunt 8 pattern-uri pe care le folosești zilnic in proiecte reale.

🕒 Debounce și throttle – stăpânește evenimentele frecvente

Evenimentele `scroll`, `resize`, `input`, `mousemove` se declanșează de zeci – sute de ori pe secunda. Daca handler-ul face ceva costisitor (fetch, recalcul layout), pagina „ingheață”.

Debounce – amana executia pana cand evenimentul nu mai apare *ms* milisecunde. Bun pentru autocomplete (aștepta sa termini de scris).

Throttle – executa cel mult o data la fiecare *ms*. Bun pentru scroll, resize.

Listing 33: Utilitar reutilizabil

```
1 function debounce(fn, ms = 300) {
2   let timer;
3   return function(...args) {
4     clearTimeout(timer);
5     timer = setTimeout(() => fn.apply(this, args), ms);
6   };
7 }
8
9 const search = document.querySelector('#search');
10 search.addEventListener('input', debounce((e) => {
11   console.log('Cautare:', e.target.value); // ruleaza dupa
12     300ms de tacere
13 }, 300));
```

Listing 34: Buton „scroll to top” care apare dupa 400px

```
1 function throttle(fn, ms = 100) {
2   let last = 0;
3   return function(...args) {
4     const now = Date.now();
5     if (now - last >= ms) { last = now; fn.apply(this, args); }
6   };
7 }
8
9 const btnTop = document.querySelector('.to-top');
10 window.addEventListener('scroll', throttle(() => {
11   btnTop.classList.toggle('visible', window.scrollY > 400);
12 }, 100));
```

🔗 Event delegation – un singur listener pentru N elemente

Daca ai 50 de butoane `.delete-btn`, NU pune 50 de `addEventListener`. Pune unul pe parinte si folosește `event.target.closest()` pentru a verifica ce a fost clicat. Bonus: funcționează și pentru elemente adaugate dinamic dupa atasare.

 continuare bonus

Listing 35: Lista de task-uri cu ștergere

```

1 const list = document.querySelector('#task-list');
2
3 list.addEventListener('click', (e) => {
4     const deleteBtn = e.target.closest('.delete-btn');
5     if (!deleteBtn) return;           // click in
        alta parte
6     deleteBtn.closest('li').remove();
7 });

```

Cum funcționează: evenimentul *bubbles* (urca de la **target** către parinti). Listener-ul pe parinte vede toate click-urile copiilor.

 Opțiuni la `addEventListener`

Parametrul al treilea poate fi un obiect cu opțiuni:

- `{ once: true }` – executa o singura data, apoi se auto-elimina. Util pentru splash/intro animation.
- `{ passive: true }` – promiti browserului ca NU vei face `preventDefault()`. Pentru `scroll/touchmove` mai ales – crește FPS pe mobile.
- `{ capture: true }` – prinde evenimentul in faza de *capture* (de sus in jos), nu *bubble* (rar folosit).

```

window.addEventListener('scroll', onScroll, { passive: true });
btn.addEventListener('click', showWelcome, { once: true });

```

 `requestAnimationFrame` – animații fluide din JS

Cand vrei sa animezi ceva din JavaScript (nu doar prin CSS), NU folosi `setTimeout(..., 16)`. Folosește `requestAnimationFrame` – se sincronizeaza cu refresh-ul ecranului (60fps natural, 120fps pe ecrane rapide).

Listing 36: Counter animat de la 0 la o valoare țintă

```

1 function animateNumber(el, target, duration = 1000) {
2     const start = performance.now();
3
4     function tick(now) {
5         const elapsed = now - start;
6         const progress = Math.min(elapsed / duration, 1); // 0..1
7         el.textContent = Math.floor(progress * target);
8         if (progress < 1) requestAnimationFrame(tick);
9     }
10
11     requestAnimationFrame(tick);
12 }
13
14 animateNumber(document.querySelector('.stat-projects'), 42);

```

 Async / await + fetch – apeluri spre API-uri

 continuare bonus

Funcțiile `async` permit scrierea codului asincron în stil sincron. `await` pauzează până când un `Promise` se rezolvă.

Listing 37: Voluntar afișează stats GitHub live pe portofoliu

```
1 async function loadGitHubStats(username) {
2   try {
3     const response = await
4       fetch('https://api.github.com/users/${username}');
5     if (!response.ok) throw new Error('HTTP
6       ${response.status}');
7
8     const data = await response.json();
9
10    document.querySelector('.gh-repos').textContent =
11      data.public_repos;
12    document.querySelector('.gh-followers').textContent =
13      data.followers;
14    document.querySelector('.gh-avatar').src =
15      data.avatar_url;
16  } catch (err) {
17    console.warn('Nu am putut incarca stats GitHub:', err);
18  }
19 }
20 loadGitHubStats('voluntar-best');
```

Trei reguli:

- `async` înainte de `function` – funcția returnează un `Promise`.
- `await` funcționează doar în interiorul unei funcții `async`.
- Erorile asincrone se prind cu `try/catch` la fel ca cele sincrone.

☰ Metode pe array – map, filter, find

În loc de `for` cu indice și `array.push()`, JavaScript oferă metode care returnează un array nou (imuabilitate). Mult mai citibile pentru transformări de date.

Listing 38: Lista de proiecte randată din date

```
1 const proiecte = [
2   { titlu: 'Portfolio', link: '/portfolio', tag: 'web' },
3   { titlu: 'Game of Life', link: '/gol', tag: 'algo' },
4   { titlu: 'Calculator', link: '/calc', tag: 'web' },
5 ];
6
7 // map -- transforma fiecare element
8 const html = proiecte
9   .filter((p) => p.tag === 'web') // doar
10    proiectele web
11   .map((p) => '<li><a href="${p.link}">${p.titlu}</a></li>');
```

continuare bonus

```

11     .join('');
12
13     document.querySelector('.projects').innerHTML = html;
14
15     // find -- gasește textul care este primul element care
16     // satisface
17     const portfolio = proiecte.find((p) => p.titlu === 'Portfolio');

```

reduce (mai rar în front-end de început) acumulează un singur rezultat din array (suma, grupări, statistici).

Destructuring + template literals

Două sintaxe pe care le vezi peste tot. Destructuring extrage valori din obiecte/array-uri într-un singur pas:

```

const { nume, varsta } = persoana;           // în loc de
      persoana.nume, persoana.varsta
const [primul, al_doilea] = limbaje;        // în loc de
      limbaje[0], limbaje[1]

// Aplicat în parametri de funcție
function saluta({ nume, varsta }) {
    return `${nume}, ${varsta} ani`;
}
saluta({ nume: 'Voluntar', varsta: 21 });

```

Template literals (backtick) permit interpolare `${...}` și string-uri multi-linie. Le-ai folosit deja în exemplul GitHub și în HTML-ul generat din `map`.

localStorage – salvează preferințele user-ului

localStorage este un dicționar persistent între vizite. Stochează doar string-uri – pentru obiecte folosești `JSON.stringify/JSON.parse`.

Listing 39: Tema dark se păstrează între vizite

```

1  const toggle = document.querySelector('.toggle-theme');
2
3  // La încărcare: aplică tema salvată
4  if (localStorage.getItem('theme') === 'dark') {
5      document.body.classList.add('dark');
6  }
7
8  // La click: schimbă și salvează
9  toggle.addEventListener('click', () => {
10     document.body.classList.toggle('dark');
11     const tema = document.body.classList.contains('dark') ?
12         'dark' : 'light';
13     localStorage.setItem('theme', tema);
14 });

```

sessionStorage e identic, dar se șterge când userul închide tab-ul. Ambele au limită 5MB per

🔗 continuare bonus

origin.

📍 Unde mergi de aici

- MDN `IntersectionObserver`: https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API
- `javascript.info` – Promises, async/await: <https://javascript.info/async>
- `web.dev` Event handling best practices: <https://web.dev/articles/optimize-input-delay>
- Free public APIs pentru exersat: <https://github.com/public-apis/public-apis>

Verificare cunoștințe

☰ PROVOCARE

1. Ce face atributul `defer` pe un `<script>` și de ce este necesar?
2. Care sunt cei trei pași ai pattern-ului **SELECT** → **LISTEN** → **CHANGE**?
3. De ce `===` este preferabil lui `==`? Dați un exemplu de coerciție surprinzătoare.
4. Ce face `e.preventDefault()` apelat într-un handler de `submit`?
5. Spot the issue: `const btn = document.querySelector('.cta');`
`btn.addEventListener(...);` – `btn` este `null`.
6. De ce este `IntersectionObserver` preferabil lui `scroll` listener pentru animații la scroll?
7. Ce face `observer.unobserve(el)` și când îl folosești?

Recapitulare

✅ FAZA 3 – JAVASCRIPT

- Includere: `<script src="..." defer></script>` în `<head>`.
- `const` este declararea implicită; `let` doar când reassignarea este necesară; nu `var`.
- Comparatie: exclusiv `===`, niciodată `==`.
- Pattern-cheie: **SELECT** → **LISTEN** → **CHANGE** acopera majoritatea interacțiunilor.
- DOM: `querySelector` (un element); `querySelectorAll` (NodeList iterabil cu `.forEach`).
- Form submit: `e.preventDefault()` + `new FormData(form) + Object.fromEntries(...)`.
- Cache referintele DOM în variabile; nu repeta `querySelector` pentru același element.
- `IntersectionObserver` pentru animații la scroll: zero cost de performance, callback doar când vizibilitatea se schimbă.
- Debug: `F12` → `Console`, `console.log()`, `debugger`.
- 🧠 Bonus: `debounce/throttle`, event delegation, `requestAnimationFrame`, `fetch` cu `async/await`, `map/filter/find`, destructuring, `localStorage`.

🔧 APLICAȚIE PRACTICĂ

Platforma: CodePen (codepen.io). URL-ul exact este distribuit de trainer in chat.

Activitatea. Documentul de pornire contine HTML si CSS complet, dar fisierul JavaScript este gol. Sarcina este implementarea pattern-ului SELECT-LISTEN-CHANGE: la apasarea butonului prezent in pagina, clasa `dark` se aplica si se scoate alternativ pe elementul `<body>`.

Solutia asteptata:

```
const btn = document.querySelector('button');
btn.addEventListener('click', () => {
  document.body.classList.toggle('dark');
});
```

Durata estimata: 5 minute.

🎓 Resurse pentru aprofundare

- Kantor I. *The Modern JavaScript Tutorial*. <https://javascript.info>
- Mozilla Developer Network. *JavaScript reference*.
<https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- The Odin Project. *JavaScript Foundations*.
<https://www.theodinproject.com/paths/foundations/courses/foundations#javascript-basics>
- Mozilla Developer Network. *Document Object Model*.
https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- Google. *Learn JavaScript*. <https://web.dev/learn/javascript>

→ **Urmeaza:** Capitolul 4 (Deploy) muta site-ul de pe mașina locala pe un URL public. Toate fazele 1-3 sunt locale; abia dupa deploy pagina exista in lume.

Faza 4 – Deploy

Versionare cu git. Publicare prin GitHub Pages.

Capitolul descrie procesul de publicare a unui site web static prin combinația git – GitHub – GitHub Pages. Acoperirea include comenzile esențiale ale fluxului de lucru, configurarea unui repository remote, activarea serviciului de hosting integrat, și convenii de scriere a mesajelor de commit.

🕒 CE ÎNVEȚI

Dupa capitolul asta vei ști:

- Executa cele 5 comenzi git pentru inițializarea unui repository și primul push.
- Configurează GitHub Pages pe un repository pentru hosting static automat.
- Folosește `git status`, `git diff`, `git log` pentru inspectarea stării.
- Scrie mesaje de commit la imperativ, descriptive, sub 72 caractere.
- Excludere fișiere sensibile prin `.gitignore`.

❓ DE CE

Site-ul construit local pe mașina dezvoltatorului nu este accesibil nimănui altcuiva. Deploy-ul transformă cod local în URL public. În plus, git este standardul industrial pentru versionare – cunoștințele dobândite aici se aplică identic în orice job de dezvoltare software. Mesajele de commit bune sunt diferența între un istoric care ajută la debug și unul inutil.

📖 PRE-RECHIZITE

Capitolul presupune un site funcțional în localhost (Faza 1–3) și un cont GitHub creat în prealabil.

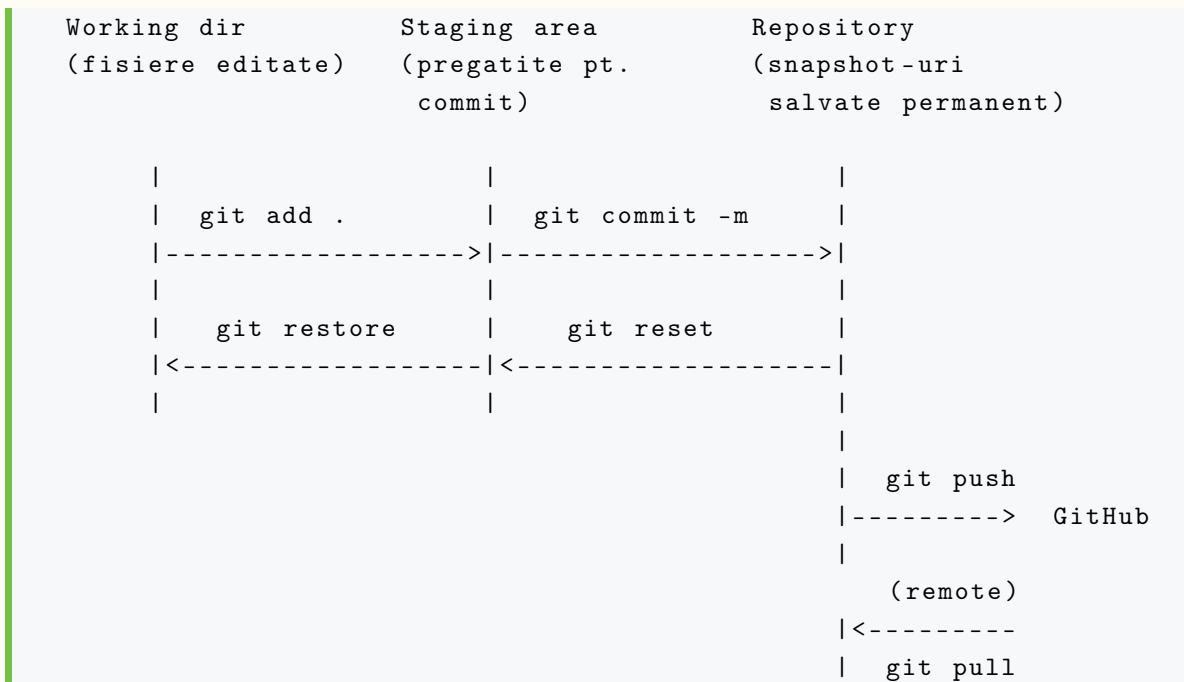
1. De ce git?

Git este sistemul de control al versiunilor (*Version Control System*) folosit de aproape toată industria software. Beneficiile față de o soluție de tip „upload manual prin FTP”:

- **Istoric.** Fiecare commit reprezintă un *snapshot* al codului, identificat printr-un hash unic. Reverența la o versiune anterioară este o operație atomică.
- **Backup distribuit.** Repository-ul există local, pe GitHub, și pe orice copie clonată. Pierderea unei singure copii nu afectează istoricul.
- **Colaborare.** Mai mulți autori pot lucra simultan pe ramuri (*branches*) separate, sincronizate prin merge.
- **Standard industrial.** Cunoașterea git este o cerință de bază în posturile de dezvoltare software.

2. Modelul mental: trei zone

Git menține trei zone în care fișierele pot exista la un moment dat:



- **Working directory** – fișierele așa cum apar pe disc.
- **Staging area** (sau *index*) – multimea fișierelor pregătite pentru următorul commit.
- **Repository** – istoricul snapshot-urilor confirmate.

Comenzile `add`, `commit`, `push` muta starea între zone în direcția stanga → dreapta.

3. Cele cinci comenzi de baza

Fluxul minim pentru publicarea unui director nou pe GitHub:

```

1 git init -b main # 1
2 git add . # 2
3 git commit -m "Initial commit" # 3
4 git remote add origin URL_DE_PE_GITHUB # 4
5 git push -u origin main # 5

```

- `git init -b main` – initializează un repository local cu ramura implicită `main`.
- `git add .` – include toate fișierele modificate în zona de pregătire (*staging area*).
- `git commit -m "..."` – creează un snapshot al stării pregătite, cu un mesaj descriptiv.
- `git remote add origin ...` – asociază repository-ul local cu un remote pe GitHub.
- `git push -u origin main` – trimite commit-urile locale către remote, stabilind ramura `main` ca *upstream*.

4. Comenzi de inspectie (read-only)

Înainte de `commit`, două comenzi confirmă ce urmează să fie salvat:

Listing 40: Verificare inainte de commit

```

1 git status           # Lista fisierelor modificate / pregatite
2 git diff             # Diferenta linie cu linie (working dir)
3 git diff --staged    # Diferenta in staging area
4 git log --oneline -10 # Ultimele 10 commit-uri, format compact

```

Aceste comenzi nu modifica nimic; pot fi rulate oricand fara risc.

5. Configurarea GitHub Pages

Procedura standard pentru activarea hosting-ului:

1. Creare a unui repository public la <https://github.com/new>, fara fisiere implicate (README, gitignore).
2. Copierea URL-ului HTTPS afisat de GitHub si executarea comenzii `git remote add origin <URL>` urmata de `git push -u origin main`.
3. Navigare la *Settings* → *Pages* (in panoul lateral al repository-ului).
4. Selectarea ramurii `main` si a folder-ului `/ (root)` in sectiunea *Branch*, urmata de *Save*.
5. Asteptarea de 30 – 60 secunde, dupa care URL-ul public devine accesibil in caseta verde din partea de sus.

Anatomia URL-ului public: <https://USERNAME.github.io/REPO>

6. Workflow zilnic dupa primul push

Actualizarea continutului dupa deploy-ul initial necesita doar trei comenzi:

```

1 git status           # ce s-a modificat?
2 git add .            # marcare pentru commit
3 git commit -m "Adauga sectiunea proiecte"
4 git push             # trimitere pe GitHub

```

GitHub Pages reconstruieste si republica pagina automat, in aproximativ 60 de secunde.

7. Convenii pentru mesajele de commit

Mesajul de commit ramane in istoricul repository-ului permanent. Calitatea acestuia este vizibila la `git log` si afecteaza utilitatea istoricului ca documentatie a evolutiei proiectului.

✖ ASA NU

```

git commit -m "update"
git commit -m "fix"
git commit -m "asdf"
git commit -m "Modificari"
git commit -m "."

```

Probleme. Niciun context. La revizuire, autorul nu mai stie ce contine fiecare commit. Inutilizabil

pentru `git revert`, `git bisect`, code review.

✓ ASA DA

```
git commit -m "Adauga sectiunea hero cu CTA"
git commit -m "Fix overflow horizontal pe mobil < 360px"
git commit -m "Implementeaza dark mode prin light-dark()"
git commit -m "Reduce dimensiunea hero.jpg de la 2.4MB la 180KB"
```

Reguli simple. (1) Verb la imperativ in prima pozitie (*Adauga*, *Fix*, *Implementeaza*, *Reduce*). (2) Maxim 50–72 caractere. (3) Descrie *ce si de ce*, nu *cum* (codul arata cum).

8. Fisierul .gitignore

Anumite fisiere si directoare nu trebuie sa ajunga in repository: dependinte instalate (`node_modules/`), fisiere temporare ale editoarelor, secretele (`.env`). Lista lor se declara in fisierul `.gitignore` la radacina proiectului:

Listing 41: Exemplu minimal pentru proiect web static

```
1 # Dependinte
2 node_modules/
3
4 # Editoare
5 .vscode/
6 .idea/
7 *.swp
8
9 # Sistem
10 .DS_Store
11 Thumbs.db
12
13 # Secrete
14 .env
15 .env.local
16
17 # Build output
18 dist/
19 build/
```

Fisierul se commit-eaza ca orice alt fisier. Toate caile listate sunt ignorate la `git add`.

⚠ ATENȚIE

Nu commite niciodata fisiere `.env` cu chei API, parole, token-uri. Odata commit-ate, raman in istoric chiar dupa stergere; pot fi recuperate de oricine cu acces la repository. Daca s-a intamplat: regenerati cheia compromisa imediat la furnizorul respectiv.

9. Probleme frecvente

⚠ ATENȚIE

Autentificare prin parola. GitHub a renunțat la autentificarea prin parola pentru operațiunile git (din 2021). Soluția recomandată este generarea unui *Personal Access Token* (PAT) la <https://github.com/settings/tokens> cu scope-ul `repo`, folosit ulterior în locul parolei. Tokenul este memorat de *Git Credential Manager* (Windows) sau *Keychain* (macOS), nu de browser – la urmatorul `git push` nu va mai fi cerut.

⚠ ATENȚIE

Eroare 404 pe pagina deployata. Cauze posibile: (1) intervalul de propagare nu a fost respectat (60 secunde după *Save*); (2) fișierul de pornire nu se numește `index.html` (este sensibil la majuscule pe serverul GitHub); (3) cache-ul browserului afișează versiunea precedentă – se rezolvă cu `Ctrl+F5`.

Verificare cunoștințe

☰ PROVOCARE

1. Care sunt cele 5 comenzi git pentru inițializare și primul push?
2. De ce GitHub a renunțat la autentificarea prin parola? Care este alternativa?
3. *Spot the issue:* `git commit -m "update"`.
4. Diferența între `git status` și `git diff`?
5. Pentru un repository numit `portfolio` al utilizatorului `voluntar-best`, care este URL-ul GitHub Pages rezultat?

Recapitulare

✓ FAZA 4 – DEPLOY

- Modelul mental: `working dir` → `staging` → `repository` → `remote`.
- Cinci comenzi de baza: `git init -b main`, `git add .`, `git commit -m`, `git remote add origin`, `git push -u origin main`.
- Comenzi de inspectie: `git status`, `git diff`, `git log -oneline -10`.
- Activare Pages: *Settings* → *Pages* → branch `main`, folder / (root) → *Save*.
- Mesaje de commit: verb la imperativ, ≤72 caractere, descrie *ce* și *de ce*.
- Workflow zilnic: 3 comenzi (`add`, `commit`, `push`); redeploy automat în ~60s.
- Autentificare prin Personal Access Token (PAT), nu prin parola.
- `.gitignore` exclude `node_modules/`, `.env`, fișiere de editor, build artifacts.

• • •

🔗 APLICAȚIE PRACTICĂ

Platforma: GitHub (<https://github.com>) – aplicația practică este deploy-ul efectiv al site-ului construit în fazele 1–3.

Activitatea. Procedura completa, executata sincron de toti participantii sub indrumarea trainer-ului:

1. Inchiderea fisierelor in editor; navigare in *Git Bash* la directorul proiectului.
2. Executarea celor cinci comenzi git din sectiunea „Cele cinci comenzi de baza”.
3. Crearea repository-ului public pe GitHub si conectarea remote-ului.
4. Activarea GitHub Pages in setari.
5. Verificarea URL-ului public si publicarea acestuia in canalul Discord al grupului.

Durata estimata: 10 minute.

Resurse pentru aprofundare

- GitHub Inc. *GitHub Pages documentation*. <https://pages.github.com>
- Cottle P. *Learn Git Branching*. <https://learngitbranching.js.org>
- Chacon S., Straub B. *Pro Git* (editia a 2-a). <https://git-scm.com/book/en/v2>
- GitHub Inc. *Configuring a custom domain for your GitHub Pages site*.
<https://docs.github.com/pages/configuring-a-custom-domain-for-your-github-pages-site>
- Beams C. *How to Write a Git Commit Message*. <https://cbea.ms/git-commit/>
- Alternative: <https://www.netlify.com>, <https://vercel.com> (servicii similare cu deploy automat din git).

→ **Urmeaza:** Capitolul 5 (SEO) face site-ul descoperibil. URL public exista; acum trebuie ca motoarele de cautare și rețelele sociale sa-l afișeze corect.

Faza 5 – SEO + Open Graph

Metadate pentru motoarele de cautare si rețelele sociale.

Capitolul prezinta tag-urile meta utilizate de motoarele de cautare (in principal Google) si de platformele sociale (Facebook, X, LinkedIn, WhatsApp, Discord) pentru afisarea corecta a paginilor distribuite. Acoperirea include SEO clasic, protocolul Open Graph (OGP), datele structurate Schema.org, si URL-urile canonice.

🎯 CE ÎNVEȚI

Dupa capitolul asta vei ști:

- Configureaza `<title>` și `<meta description>` pentru rezultatele de cautare.
- Aadauga cele 5 tag-uri `og:*` pentru previzualizare in rețele sociale.
- Genereaza și valideaza imaginea `og:image` la dimensiunile și formatul corect.
- Aadauga date structurate (JSON-LD) pentru rich snippets in Google.
- Genereaza `robots.txt` și `sitemap.xml` pentru a ghida crawler-ele.
- Distinge mit-urile SEO de practica actuala.
- 🦋 *Bonus self-study:* tipuri Schema.org (Article, BreadcrumbList, FAQ), `preconnect/dns-prefetch`, linking intern, mobile-first indexing.

❓ DE CE

Un site fara SEO este invizibil. Aplicațiile sociale moderne (WhatsApp, Discord, LinkedIn) afișează by default cardul de previzualizare; un site fara meta tags arata neprofesional cand cineva il share-uieste. SEO de baza este *configurare*, nu *magic*: cele 7-10 tag-uri prezentate aici acopera 80% din optimizarea esentiala.

ℹ️ PRE-RECHIZITE

Capitolul presupune un site deployat (Faza 4) cu URL public accesibil. Tag-urile se valideaza pe varianta live, nu pe localhost.

1. Doua audiente, doua seturi de tag-uri

- **Motoare de cautare** (Google, Bing). Folosesc `<title>`, `<meta name="description">`, structura semantica, datele Schema.org.
- **Platforme sociale** (Facebook, X, LinkedIn, WhatsApp, Discord). Folosesc protocolul Open Graph: `<meta property="og:*">`.

2. Metadate pentru motoarele de cautare

Doua elemente determina aspectul paginii in rezultatele cautarii:

Listing 42: Plasare in `<head>`

```
<title>Voluntar BEST -- Student CS, Iasi</title>
```

```
2 <meta name="description" content="Pagina personala a Voluntarului
   BEST, studenta la CS, pasionata de web development.">
```

2.1 Calitatea <title>

✖ ASA NU

```
<title>Document</title>
<title>Untitled</title>
<title>Index</title>
<title>Acasa</title>
<title>web web web html portofoliu cv student iasi cs
  uaic</title>
```

Probleme. Generic, nesemnificativ; sau spam de cuvinte-cheie penalizat de Google.

✔ ASA DA

```
<title>Voluntar BEST -- Portofoliu Web Developer, Iasi</title>
<title>Despre BEST Iasi -- Asociatie studenteasca tehnica</title>
```

Reguli. 50–60 caractere; specific paginii (nu site-ului); structura „Subiect – Context”.

2.2 Calitatea <meta description>

Aceleasi reguli ca pentru <title>, doar lungimea difera: **150–160 caractere**, descrie conținutul concret (proiecte, CV, secțiuni), fara fraze de umplutura tip „Bine ați venit”.

```
<meta name="description" content="Portofoliul Voluntarului BEST:
  proiecte web (HTML, CSS, JS), CV, contact. Voluntar BEST Iasi si
  student CS la UAIC.">
```

3. Protocolul Open Graph

Open Graph Protocol (OGP), specificat de Facebook in 2010, este standardul de fapt pentru previzualizarea paginilor web in rețelele sociale. Implementarea consta in adaugarea a cinci tag-uri <meta> cu prefixul og::

Listing 43: Tag-uri OGP minimale

```
1 <meta property="og:type" content="website">
2 <meta property="og:title" content="Voluntar BEST -- Student CS,
   Iasi">
3 <meta property="og:description" content="Pagina personala. Web dev,
   design, open source.">
4 <meta property="og:image"
   content="https://voluntar-best.github.io/site/og.png">
5 <meta property="og:url"
   content="https://voluntar-best.github.io/site/">
```


- `og:type` – categoria conținutului. Valoarea `website` acopera majoritatea cazurilor; alternative: `article`, `video`, `book`, `profile`.
- `og:title` și `og:description` – pot diferi de `<title>` și `<meta description>`, fiind optimizate specific pentru context social (mai punctuale, cu CTA).
- `og:image` – dimensiunea recomandată 1200 x 630 pixeli (raport 1.91:1).
- `og:url` – URL-ul canonic al paginii.

3.1 Constrângerea `og:image`: URL absolut

✘ ASA NU – URL RELATIV

```
<meta property="og:image" content="og.png">
<meta property="og:image" content="/images/og.png">
```

Problema. Crawler-ele platformelor sociale nu pot rezolva caile relative, întrucât nu cunosc contextul de bază al paginii sursă. Cardul afișat rămâne fără imagine.

✔ ASA DA – URL ABSOLUT

```
<meta property="og:image" content="https://voluntar-best.github.io/site/og.png">
```

Beneficiu. Imaginea este accesibilă direct prin URL și poate fi cache-uită de Facebook, Slack, Discord.

3.2 Calitatea imaginii `og:image`

- Dimensiune ideală: **1200 x 630 px** (raport 1.91:1, formatul afișat de Facebook și LinkedIn).
- Dimensiune minimă: 600 x 315 px. Sub această limită, unele platforme nu afișează cardul mare.
- Format: JPG sau PNG; dimensiunea maximă 8 MB (Facebook), în practică ≤ 1 MB pentru încărcare rapidă.
- Conținut lizibil la dimensiune redusă: titlul + 1–2 elemente vizuale, nu paragrafe întregi.

4. Twitter Card

Platforma X (anterior Twitter) suportă nativ OGP, dar acceptă și tag-uri proprii pentru carduri cu imagine mare:

```
<meta name="twitter:card" content="summary_large_image">
```

Adăugarea acestei singure linii instruieste X să afișeze cardul în formatul extins, folosind valorile deja prezente în tag-urile `og:*`.

5. Date structurate (JSON-LD)

Schema.org definește un vocabular standardizat pentru descrierea entităților (persoane, organizații, evenimente, articole) într-un format pe care motoarele de căutare îl procesează explicit. Implementarea recomandată: JSON-LD plasat într-un `<script>` cu tipul `application/ld+json`.

Listing 44: Schema Person pentru o pagina personala

```

1 <script type="application/ld+json">
2 {
3   "@context": "https://schema.org",
4   "@type": "Person",
5   "name": "Voluntar BEST",
6   "url": "https://voluntar-best.github.io/site/",
7   "jobTitle": "Student CS",
8   "alumniOf": "Universitatea Alexandru Ioan Cuza",
9   "sameAs": [
10    "https://github.com/voluntar-best",
11    "https://www.linkedin.com/in/voluntar-best"
12  ]
13 }
14 </script>

```

Avantajul principal: Google poate afisa pagina cu *rich snippets* (foto, link-uri sociale, descriere structurata) in rezultatele cautarii. Validarea se face cu Google Rich Results Test (<https://search.google.com/test/rich-results>).

🔗 PONT

Pentru un eveniment, foloseste "@type": "Event" cu campurile `startDate`, `location`, `organizer`. Google poate afisa evenimentul direct intr-un panel lateral. Vocabularul complet: <https://schema.org>.

6. URL-ul canonic

Cand acelasi continut este accesibil prin mai multe URL-uri (cu si fara `www`, cu si fara slash final, cu parametri UTM, etc.), Google trebuie sa stie care URL este versiunea „oficiala”. Tag-ul canonical previne penalizarea pentru continut duplicat:

```

1 <link rel="canonical" href="https://voluntar-best.github.io/site/">

```

Indicatie: pe paginile de proiect, URL-ul canonic este URL-ul „curat” al paginii, fara parametri de tracking.

7. Mituri SEO

- **Mit:** `<meta name="keywords">` influenteaza ranking-ul.
- **Realitate:** Google a anuntat oficial in 2009 ca ignora complet acest tag. Bing si Yandex il folosesc minim. **A nu se mai folosi.**
- **Mit:** repetarea cuvintelor cheie in continut creste ranking-ul.
- **Realitate:** *keyword stuffing* este penalizat. Algoritmi modernii (BERT, MUM) inteleg sensul textului si premiaza scrierea naturala.
- **Mit:** site-urile noi sunt penalizate de Google.
- **Realitate:** indexarea dureaza 1–7 zile pentru un site nou. Inscrierea in Google Search Con-

sole (<https://search.google.com/search-console>) accelereaza procesul.

8. robots.txt — ghideaza crawler-ele

`robots.txt` este un fisier text plasat in radacina site-ului (<https://voluntar-best.github.io/robots.txt>) care indica robotilor de crawl ce pagini sa indexeze si unde sa caute sitemap-ul. Format strict, dar simplu:

Listing 45: robots.txt minimal pentru GitHub Pages

```
User-agent: *
Allow: /

Sitemap: https://voluntar-best.github.io/sitemap.xml
```

Cum se citește:

- **User-agent: *** – regulile se aplica oricarui robot (Googlebot, Bingbot, etc.).
- **Allow: /** – permite accesul la intreg site-ul.
- **Disallow: /admin/** (alternativ) – interzice anumite cai. Util pentru pagini in lucru, dashboard-uri private.
- **Sitemap: ...** – URL absolut catre sitemap-ul XML (vezi sectiunea urmatoare).

⚠ ATENȚIE

`robots.txt` este o *conventie*, nu un mecanism de securitate. Crawler-ele rau-voitoare il ignora. Daca ai date sensibile, foloseste autentificare reala, nu `Disallow`.

💡 PONT

Verificare: deschide direct in browser <https://<site>/robots.txt>. Daca se afiseaza textul, e plasat corect.

9. sitemap.xml — ce pagini exista

Sitemap-ul este un fisier XML care listeaza toate paginile importante ale site-ului. Google il foloseste pentru indexare mai rapida si completa, in special pentru site-urile noi sau cu structura adanca.

Listing 46: sitemap.xml scris manual pentru un portofoliu mic

```
<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  <url>
    <loc>https://voluntar-best.github.io/</loc>
    <lastmod>2026-05-11</lastmod>
    <priority>1.0</priority>
  </url>
  <url>
    <loc>https://voluntar-best.github.io/projects.html</loc>
    <lastmod>2026-05-08</lastmod>
    <priority>0.8</priority>
```

```

</url>
<url>
  <loc>https://voluntar-best.github.io/contact.html</loc>
  <lastmod>2026-04-22</lastmod>
  <priority>0.5</priority>
</url>
</urlset>

```

Reguli:

- `<loc>` – URL absolut (obligatoriu).
- `<lastmod>` – data ultimei modificari, format ISO YYYY-MM-DD (opțional, recomandat).
- `<priority>` – 0.0 – 1.0, relativ in cadrul site-ului. Home = 1.0, pagini interne = 0.5–0.8.
- Pentru site-uri mici (<50 pagini), scrierea manuala este OK. Pentru site-uri mari, foloseste generatoare automate.

9.1 Submisie in Google Search Console

1. Deschide <https://search.google.com/search-console>, adauga proprietatea (URL prefix: <https://voluntar-best.github.io/>).
2. Verifica posesiunea – pentru GitHub Pages, metoda recomandata e meta tag in `<head>`.
3. Secțiunea „Sitemaps” → adauga [sitemap.xml](#) → Submit.
4. In 1–7 zile, Google va incepe sa indexeze toate URL-urile listate.

🔗 PONT

Beneficiul real: fara sitemap, Google poate descoperi pagini doar prin link-uri. Cu sitemap, le **declari explicit**. Pentru un site nou (fara backlink-uri inca), diferența e intre indexare in 7 zile vs in 1–2 zile.

10. Favicon

Favicon-ul (iconita afisata in tab-ul browserului si in bookmark-uri) se declara prin elementul `<link>`:

```

1 <link rel="icon" href="favicon.svg" type="image/svg+xml">
2 <link rel="apple-touch-icon" href="apple-touch-icon.png">

```

Generarea unui favicon simplu se poate face cu serviciul <https://favicon.io/favicon-generator/>, care produce fisierele necesare pornind de la o singura litera si o culoare. Pentru un favicon profesional din SVG, conversia automata in toate dimensiunile necesare se face cu <https://realfavicongenerator.net>.

🔗 BONUS • AVANSAT

Acest box e optional. Daca esti la inceput, treci direct la *Verificare*. Mai jos sunt pattern-urile pe care le folosesc site-urile cu trafic real.

🏠 Tipuri Schema.org dincolo de Person

 continuare bonus

Vocabularul Schema.org are sute de @type-uri. Cele mai utile in practica:

Article – pentru blog/posts:

```
{
  "@context": "https://schema.org",
  "@type": "Article",
  "headline": "Cum am invatat web development in 2 saptamani",
  "author": { "@type": "Person", "name": "Voluntar BEST" },
  "datePublished": "2026-05-11",
  "dateModified": "2026-05-11",
  "image":
    "https://voluntar-best.github.io/blog/learning/cover.jpg"
}
```

BreadcrumbList – navigare ierarhică in rezultate:

```
{
  "@context": "https://schema.org",
  "@type": "BreadcrumbList",
  "itemListElement": [
    { "@type": "ListItem", "position": 1, "name": "Acasa",
      "item": "https://voluntar-best.github.io/" },
    { "@type": "ListItem", "position": 2, "name": "Proiecte",
      "item": "https://voluntar-best.github.io/projects/" },
    { "@type": "ListItem", "position": 3, "name": "Portfolio
      Web" }
  ]
}
```

Google afișează aceasta ierarhie direct in rezultatele de cautare – click rate semnificativ mai bun.

FAQPage – întrebări frecvente, afișate ca rich snippet:

```
{
  "@context": "https://schema.org",
  "@type": "FAQPage",
  "mainEntity": [{
    "@type": "Question",
    "name": "Cum aplic la BEST Iasi?",
    "acceptedAnswer": { "@type": "Answer", "text": "Inscrie-te
      la recrutarea din toamna pe best.org.ro." }
  }]
}
```

Event – cu datele cheie indexate:

```
{
  "@context": "https://schema.org",
  "@type": "Event",
  "name": "BEST Training Web 2026",
}
```

🔗 continuare bonus

```

"startDate": "2026-11-15T10:00:00+02:00",
"location": { "@type": "Place", "name": "TUIASI" },
"organizer": { "@type": "Organization", "name": "BEST Iasi" }
}

```

🔗 preconnect și dns-prefetch – performance + SEO signal

Cand pagina ta foloseste resurse de pe alt domeniu (Google Fonts, CDN, Analytics), DNS lookup + TCP handshake adauga 100–300ms inainte ca resursa sa inceapa sa se descarce. **preconnect** face acest setup in paralel cu HTML parsing, in fundal:

```

<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com"
      crossorigin>
<link rel="dns-prefetch"
      href="https://www.google-analytics.com">

```

preconnect – DNS + TCP + TLS (cost mai mare, dar resursa e gata sa fie descarcata).

dns-prefetch – doar DNS (mai ieftin, foloseste pentru >3 domenii).

Impact: imbunatateste LCP (Core Web Vitals) cu 100–500ms cand pagina depinde de un CDN.

🔗 Linking intern – arhitectură, nu trick

Google urmareste link-urile interne pentru a intelege relevanta relativa a paginilor. Pagini cu mai multe link-uri intrand sunt considerate „mai importante”.

- Foloseste **anchor text descriptiv** ([proiectele mele web](/projects)), nu [click aici](#).
- Conecteaza paginile inrudite: pe pagina de proiect, link spre alte proiecte. Pe blog post, link catre articole inrudite.
- Evita **orphan pages** (pagini fara niciun link intrand). Google le indexeaza tarziu sau deloc.

📱 Mobile-first indexing

Din 2023, Google indexeaza **varianta mobila** a paginii, nu desktop. Pagini care arata bine pe desktop dar sunt rupte pe mobile sunt penalizate masiv. Verifica:

- `<meta name="viewport" content="width=device-width, initial-scale=1">` – obligatoriu.
- Textul lizibil fara zoom (`font-size > 14px` pe mobil).
- Butoane clicabile (>44x44px, conform Apple HIG).
- Conținutul important **nu este ascuns** pe mobil (Google considera continutul vizibil ca cel canonic).
- Aceași pagina pe mobil și desktop – nu varianta separata [m.site.ro](#).

🏆 Performance ca semnal de ranking

Din 2021, Core Web Vitals (LCP, INP, CLS – vezi Faza 6) sunt **semnal de ranking**. Doua pagini cu conținut similar: cea mai rapida castiga. Nu vei concura cu site-uri lente.

Recap rapid:

- **LCP** <2.5s – cea mai mare imagine/text apare repede.
- **INP** <200ms – pagina raspunde rapid la click/tap.

🔑 continuare bonus

- CLS <0.1 – layout-ul nu „sare” după încărcare.

📍 Unde mergi de aici

- Schema.org vocabular complet: <https://schema.org/docs/full.html>
- Generator JSON-LD vizual: <https://technicalseo.com/tools/schema-markup-generator/>
- Google Mobile-Friendly Test: <https://search.google.com/test/mobile-friendly>
- Sitemap generator pentru site-uri statice: <https://www.xml-sitemaps.com>

Verificare cunoștințe

☰ PROVOCARE

1. Câte tag-uri `og:*` sunt minim necesare pentru un card valid de previzualizare?
2. De ce `og:image` trebuie să fie URL absolut, nu relativ?
3. Spot the issue: `<title>Document</title>`.
4. Care este diferența funcțională între `<title>` și `og:title`?
5. De ce `<meta name="keywords">` este considerat depășit?
6. Ce reprezintă directiva `Sitemap:` în `robots.txt`?
7. De ce `<lastmod>` în `sitemap.xml` este util pentru Google?

Recapitulare

✔ FAZA 5 – SEO + OPEN GRAPH

- Două audiente: motoare de căutare (`<title>` + `<meta description>`) și rețele sociale (5 `og:*` tags).
- `<title>`: 50–60 caractere, specific, structura „Subiect – Context”.
- `<meta description>`: 150–160 caractere, descriere concretă, fără umplutură.
- `og:image`: URL absolut, dimensiune optimă 1200 x 630, format JPG/PNG, ≤1MB.
- `<meta name="twitter:card" content="summary_large_image">` activează cardul mare pe X.
- JSON-LD (Schema.org) descrie entități: Person, Organization, Article, Event.
- `<link rel="canonical">` previne penalizarea pentru conținut duplicat.
- `robots.txt` ghidează crawler-ele și declară sitemap-ul. `sitemap.xml` listează URL-urile importante.
- Submisia în Google Search Console accelerează indexarea pentru site-uri noi.
- `<meta name="keywords">` este ignorat de Google din 2009 – **a nu se folosi**.
- 🐼 Bonus: tipuri Schema.org (Article, BreadcrumbList, FAQ, Event), `preconnect/dns-prefetch`, linking intern, mobile-first indexing.



🔧 APLICAȚIE PRACTICĂ

Platforma: OpenGraph.xyz (<https://opengraph.xyz>).

Activitatea.

1. Adaugarea celor cinci tag-uri `og:*` în `<head>`, cu valori specifice paginii personale.
2. Inlocuirea valorii pentru `og:image` cu un URL absolut către o imagine 1200x630, accesibilă public.
3. Commit și push: `git add .; git commit -m "Adauga meta tags Open Graph"; git push.`
4. Accesarea OpenGraph.xyz și lipirea URL-ului public al paginii.
5. Verificarea previzualizărilor pentru Facebook, X, LinkedIn, WhatsApp, iMessage.

Diagnoza problemelor: (1) URL-ul `og:image` nu este absolut; (2) imaginea returnează HTTP 404; (3) dimensiunea imaginii este sub 600 pixeli pe oricare axă.

Durata estimată: 5 minute.

🎓 Resurse pentru aprofundare

- Open Graph Protocol. *Specificația oficială*. <https://ogp.me>
- Meta Tags. *Generator vizual*. <https://metatags.io>
- Google. *Rich Results Test*. <https://search.google.com/test/rich-results>
- Google. *Search Console* (indexare manuală). <https://search.google.com/search-console>
- Google. *Document metadata*. <https://web.dev/learn/html/metadata>
- Schema.org. *Structured data documentation*. <https://schema.org/docs/gs.html>
- X Inc. *Twitter Cards documentation*. <https://developer.x.com/en/docs/twitter-for-websites/cards>

→ **Urmeaza:** Capitolul 6 (Lighthouse) oferă o măsurătoare obiectivă a calității. Toate fix-urile aplicate în fazele 1–5 sunt acum cuantificate într-un scor 0–100.

Faza 6 – Lighthouse

Audit automatizat. Indicatori de performanta. Core Web Vitals.

Lighthouse este un instrument de audit automatizat dezvoltat de Google, integrat in Chrome DevTools si in serviciul web PageSpeed Insights. Capitolul prezinta cele patru categorii evaluate, modul de interpretare a raportului, indicatorii *Core Web Vitals* folositi de Google in algoritmul de ranking, si remedierile pentru problemele frecvente.

🕒 CE INVEȚI

Dupa capitolul asta vei ști:

- Ruleaza un audit Lighthouse pe propria pagina prin PageSpeed Insights sau DevTools.
- Citește raportul și identifica problemele cu impact maxim pe scor.
- Aplica fix-uri uzuale: `alt`, `lang`, `width/height` pe imagini, `defer` pe script.
- Optimizeaza imaginile cu format modern (WebP), `srcset`, `loading="lazy"`.
- Optimizeaza fonturile: `font-display: swap`, `preload`, subseturi, self-hosting.
- Înțelege *critical rendering path*: ce blocheaza prima afișare si cum reduci timpul pana la LCP.
- Interpreteaza Core Web Vitals (LCP, INP, CLS) și pragurile lor.
- 🐞 *Bonus self-study*: minification, third-party scripts, service worker, Lighthouse CI, `content-visibility`.

❓ DE CE

Lighthouse este standardul industrial pentru evaluarea unei pagini web. Scorul 95+ pe toate cele 4 categorii este criteriul implicit la code review in firmele moderne. Mai mult, Core Web Vitals influenteaza direct ranking-ul Google. Un audit de 2 minute identifica 80% din problemele care ar costa ore de cautare manuala.

📄 PRE-RECHIZITE

Capitolul presupune un site deployat (Faza 4) si configurat cu meta tags (Faza 5). Auditul ruleaza pe URL-ul public.

1. Categoriile de evaluare

Lighthouse genereaza un scor 0–100 pe fiecare dintre cele patru axe:

- **Performance** – timpul de incarcare si receptivitatea paginii.
- **Accessibility** – conformitatea cu standardele WCAG (in principal contrast, structura semantica, asocieri label–input, attribute `alt`).
- **Best Practices** – folosirea de protocoale si API-uri moderne, absenta erorilor in consola, lipsa codului depasit.
- **SEO** – prezenta meta tag-urilor obligatorii, structura semantica, indexabilitatea.

Scorurile se interpreteaza astfel: peste 90 = bun; peste 95 = nivel profesional; sub 50 = necesita

interventie.

1.1 Lab data vs field data

Lighthouse folosește *lab data*: un scenariu sintetic în condiții standardizate (CPU lent, rețea simulată 4G slow). Scorul reflectă capacitatea pe un dispozitiv mediu, nu situația reală a utilizatorilor. Pentru date din producție, PageSpeed Insights afișează și *field data* extrase din Chrome User Experience Report (CrUX), agregate pe ultimele 28 de zile.

2. Modalități de execuție

2.1 PageSpeed Insights (interfață web)

Cea mai accesibilă metodă. Pași: accesarea <https://pagespeed.web.dev>, lipirea URL-ului public al paginii, executarea analizei. Avantaje: nu necesită instalare; include datele reale de la utilizatori (CrUX) atunci când acestea există; raport partajabil prin URL.

2.2 Chrome DevTools (local)

Procedura: deschiderea paginii în Chrome, **F12** → tab *Lighthouse*, selectarea categoriilor, executarea *Analyze page load*. Avantaje: funcționează pe paginile servite de pe `localhost`, fără necesitatea de deploy; permite testare rapidă între modificări.

2.3 CLI (pentru integrare CI)

Lighthouse este disponibil ca pachet npm. Folosit în pipeline-uri CI/CD pentru a bloca deploy-urile care scad scorul sub un prag stabilit.

3. Structura raportului

Fiecare categorie include:

- Scorul numeric, evidențiat color (roșu / portocaliu / verde).
- Lista problemelor identificate, grupate pe tip. Selectarea unei probleme afișează descrierea, impactul, și pași de remediere.
- Lista verificărilor trecute cu succes.
- Secțiunea *Opportunities* (pentru Performance) listează îmbunătățirile cu economia estimată în milisecunde.

4. Probleme frecvente și remediere

4.1 Performance

- **Imagini fără dimensiuni explicite.** Setarea atributelor `width` și `height` pe `<img>` permite browserului să rezerve spațiu, prevenind *Cumulative Layout Shift*.
- **Resurse care blochează randarea.** Adăugarea atributului `defer` pe `<script>` permite parsarea HTML în paralel.
- **Imagini supradimensionate.** O imagine 2400x1600 redată la 800 pixeli irosește banda. Optimizare: <https://squoosh.app>.

- **Format imagini neoptimizat.** JPEG/PNG in locul WebP / AVIF.
- **Cache lipsa.** Resursele statice (CSS, JS, imagini) ar trebui servite cu header `Cache-Control: max-age=31536000` (1 an). *Caveat GitHub Pages:* servește cu `max-age=600` (10 minute) și nu poate fi modificat – Lighthouse va afișa avertismentul „Serve static assets with an efficient cache policy”. Pentru control real al cache-ului, folosește Netlify sau Cloudflare Pages.

4.2 Accessibility

- **Atribut `alt` lipsa pe `<img>`.** Toate imaginile relevante semantic necesita `alt`; imaginile pur decorative folosesc `alt=""`.
- **Buton fara nume accesibil.** Includerea de text in interior sau adaugarea `aria-label`.
- **Etichete lipsa pe formulare.** Asocierea `<label for>` cu `<input id>`.
- **Contrast insuficient.** Conform WCAG AA, raportul de contrast minim este 4.5:1 pentru text normal si 3:1 pentru text mare. Verificare: <https://webaim.org/resources/contrastchecker/>.

4.3 Best Practices

- **Erori in consola.** Lighthouse penalizeaza orice eroare `console.error` la incarcare. Verificare: `F12 → Console`.
- **Resurse incarcate prin HTTP (nu HTTPS).** Mixed content; rezolvat automat pe GitHub Pages.

4.4 SEO

- **Document fara `<title>`.**
- **Lipsa `<meta name="description">`.**
- **Element `<html>` fara `lang`.**
- **Linkuri fara text descriptiv** (`<a href="X">click aici</a>` este penalizat).

5. Core Web Vitals

Cei trei indicatori folositi explicit de Google in algoritmul de ranking, incepand cu 2021:

- **LCP** (*Largest Contentful Paint*) – timpul pana la afisarea celui mai mare element vizibil. Prag: <2.5 secunde.
- **INP** (*Interaction to Next Paint*) – latentă între o acțiune a utilizatorului și răspunsul vizual. Prag: <200 milisecunde.
- **CLS** (*Cumulative Layout Shift*) – măsura instabilității vizuale în timpul încărcării. Prag: <0.1 .

6. Optimizarea imaginilor

Imaginile reprezintă, în medie, 50% din dimensiunea unei pagini web. Optimizarea lor are impact direct asupra scorului Performance.

6.1 Format modern

WebP reduce dimensiunea cu 25–35% fata de JPEG, AVIF cu 50%. Conversia se face cu <https://squoosh.app> (interfata web, fara instalare).

6.2 Atribute necesare pe

✖ ASA NU

```

```

Probleme. Lipseste `alt` (a11y); lipsesc `width/height` (CLS); lipseste `loading` (download eager); o singura sursa pentru toate ecranele (banda risipita pe mobil).

✔ ASA DA – OPTIMIZAT COMPLET

```

```

Beneficii. Browserul alege rezolutia potrivita din `srcset`; imaginile sub fold sunt incarcate doar la nevoie; rezerva spatiu prin `width/height`; `alt` asigura accesibilitatea.

6.3 Lazy loading

Atributul `loading="lazy"` amana descarcarea imaginilor aflate sub *fold* (zona vizibila initial). Suportul browser este universal incepand cu 2020.

Exceptie: pentru imaginea *hero* (prima imagine din viewport), se foloseste `loading="eager"` sau atributul lipseste; `loading="lazy"` aici intarzie LCP-ul.

6.4 Squoosh workflow pas cu pas

<https://squoosh.app> face conversia + redimensionarea intr-o singura sesiune, in browser (fara upload pe server). Pași:

1. Drag-and-drop imaginea originala (JPEG 4000 x 3000, 4MB).
2. In panoul *Resize* (dreapta): seteaza latimea finala (ex. 1600px pentru hero, 800px pentru carduri).
3. Schimba formatul de output din PNG/JPEG in *WebP* sau *AVIF*.
4. Ajusteaza *Quality* (75 e un sweet spot pentru WebP). Compara vizual stanga vs dreapta.
5. Download. Repeta pentru 3–4 dimensiuni daca folosești `srcset`.

Rezultat tipic: **500KB JPEG** → **60KB WebP**. Diferența de calitate la 75% e imperceptibila pentru ochiul uman, dar Lighthouse vede economia.

7. Optimizarea fonturilor

Fonturile custom (Google Fonts, fonturi cumparate) sunt al doilea cost major dupa imagini. Un `<h1>` care așteapta sa se incarce fontul = LCP intarziat. Trei tehnici care rezolva 90% din probleme.

7.1 font-display: swap

Implicit, textul cu font custom e **invizibil** pana cand fontul se descarca (*Flash of Invisible Text, FOIT*).

`font-display: swap` il afișeaza imediat cu fontul fallback, apoi schimba cand fontul custom e gata (*Flash of Unstyled Text, FOUT*).

Listing 47: Self-hosting cu font-display

```
1 @font-face {  
2   font-family: 'Lato';  
3   src: url('/fonts/lato-regular.woff2') format('woff2');  
4   font-weight: 400;  
5   font-style: normal;  
6   font-display: swap;  
7 }
```

Pentru Google Fonts, parametrul vine in URL: `?family=Lato&display=swap`.

7.2 preload pentru fonturi critice

Browserul descopera fonturile abia dupa ce parseaza CSS-ul si gaseste o regula care le folosește – prea tarziu pentru LCP. `preload` le aduce in paralel cu HTML-ul:

```
<link rel="preload" href="/fonts/lato-regular.woff2" as="font"  
      type="font/woff2" crossorigin>
```

⚠ ATENȚIE

Preload doar fonturile **folosite in viewport-ul inițial**. Preload-ul pentru fonturi rar folosite (ex. font pentru cod, vizibil doar pe pagina de blog) face contrariul: ocupa banda inainte sa fie necesar.

7.3 Subseturi și self-hosting

Fontul implicit Lato de la Google include 800+ glife (chineza, japoneza, simboluri). Daca ai nevoie doar de **latin** (engleza + romana), trimiți pe wire 5x mai mult decat nevoie. Soluții:

- **Google Fonts URL cu subset:** `?family=Lato&subset=latin-ext&display=swap`.
- **Self-hosting** cu Google Webfonts Helper: <https://gwfh.mranftl.com>. Descarci fișierele `.woff2` subsetate și CSS-ul gata configurat, le servești din site-ul tau.

Beneficii self-hosting: o solicitare HTTP mai puțin (nu mai contactezi fonts.googleapis.com), niciun cost DNS, control complet asupra cache-ului.

8. Critical rendering path – ce blocheaza prima afișare

Critical rendering path (CRP) este secvența de pași pe care browserul ii face de la primirea HTML-ului pana la afișarea pixelilor pe ecran. Doua resurse blocheaza vizibil acest path:

- **CSS** – blochează *render*. Browserul nu desenează nimic până când nu a parsat tot CSS-ul referit în `<head>`. De aceea CSS-ul critic merge `<head>`, nu la sfârșitul `<body>`.
- **JavaScript fara `defer/async`** – blochează și *parsing* și *render*. Browserul oprește totul, descarcă, execută, apoi continuă.

Listing 48: Tipare

```

1 <head>
2   <!-- BLOCANT: dar CSS-ul e critic, e ok aici -->
3   <link rel="stylesheet" href="style.css">
4
5   <!-- NEBLOCANT: descarca paralel, executa dupa parsare -->
6   <script src="app.js" defer></script>
7
8   <!-- INDEPENDENT (analytics): paralel, executie cand vrea -->
9   <script src="https://analytics.com/script.js" async></script>
10 </head>

```

8.1 De ce `defer` e default-ul pentru JS-ul tau

- `defer` – descarcă în paralel, execută *dupa* parsare HTML, în **ordinea declarată**. DOM-ul există când rulează.
- `async` – descarcă în paralel, execută *imediat ce e disponibil* (poate întrerupe parsing). Ordine non-deterministă. Pentru analytics da, pentru codul tau nu.
- Fără nimic – blochează totul. Antipattern din 2015 încoace.

8.2 Reducerea FCP / LCP

- Minimiza CSS-ul critic (tot ce e în viewport-ul inițial). 50 KB CSS = 2x mai lent pe 3G.
- Preload pentru fontul + imaginea hero.
- Inline-uiă CSS-ul critic în `<style>` pe pagini cu trafic mare (landing-uri); restul `<link>` `async`.
- Evită render-blocking 3rd party (chat widgets, Google Tag Manager) în `<head>`.

9. Îmbunătățiri suplimentare pentru Performance

9.1 Preload pentru resurse critice

Resursele critice (font principal, imaginea hero) pot fi declarate cu `<link rel="preload">` pentru ca browserul să le descarce înainte de a parsă CSS-ul.

```

1 <link rel="preload" href="fonts/lato.woff2" as="font"
2   type="font/woff2" crossorigin>
3 <link rel="preload" href="hero.webp" as="image">

```

9.2 CSS critic inline

Pentru paginile cu prima vizita prioritara (landing pages), CSS-ul critic (necesar pentru randarea zonei vizibile) poate fi inclus direct in `<head>` prin `<style>`, restul fiind incarcat asincron. Tehnica nu este necesara pentru pagini personale; merita atentie pentru site-uri comerciale.

🔗 BONUS • AVANSAT

Acest box e optional. Daca esti la inceput, treci direct la *Verificare*. Mai jos sunt optimizările pe care le faci cand vrei 100/100/100/100, nu doar 95+.

⚡ Minification – elimini spațiile, păstrezi semnificația

Codul scris pentru oameni (indentare, comentarii, spații) ocupa 30–50% in plus față de ce trebuie sa primeasca browserul. *Minification* elimina tot ce e doar pentru lizibilitate. Rezultat: fișier identic funcțional, mai mic.

Tool-uri online (zero setup):

- CSS: <https://www.toptal.com/developers/cssminifier>
- JS: <https://www.toptal.com/developers/javascript-minifier>
- HTML: <https://www.willpeavy.com/tools/minifier/>

Pentru proiecte reale, foloseste un build tool (Vite, esbuild, Parcel) care minifica automat la build. Pentru un portofoliu pe GitHub Pages, varianta online + commit la finalul fazei e suficient.

🔊 Third-party scripts – cel mai mare cost ascuns

Analytics, chat widgets, embed-uri YouTube/Twitter – fiecare adauga 50–500ms si penalty pe Best Practices.

Strategii (in ordinea creșterii impactului):

- **async pe analytics:** `<script async src="..."></script>`. Default deja pentru Google Analytics modern.
- **Lazy load la interacțiune:** chat widget se incarca **doar dupa ce user-ul dă click**. Pana atunci, un buton placeholder de 1KB.
- **Facade pentru embed-uri:** YouTube embed = 500KB. Inlocuiește cu o imagine + buton play, incarca iframe-ul la click. Tool: <https://github.com/paulirish/lite-youtube-embed>.
- **Self-host pentru analytics:** Plausible/Umami self-hosted in loc de Google Analytics. Zero penalty Lighthouse, GDPR-friendly.

Listing 49: Lazy load pentru chat widget

```
1 const chatBtn = document.querySelector('.chat-launcher');
2 chatBtn.addEventListener('click', () => {
3   const script = document.createElement('script');
4   script.src = 'https://widget.intercom.io/widget/abc123';
5   document.head.appendChild(script);
6 }, { once: true });
```

📶 content-visibility: auto – skip rendering pe ce nu e vizibil

CSS modern (2023+) care permite browserului sa nu randeze elementele aflate sub fold pana cand utilizatorul scroll-eaza catre ele. Impact major pe pagini lungi (blog posts, documentari).

```
.long-section { content-visibility: auto;
  contain-intrinsic-size: 500px; }
```

continuare bonus

`contain-intrinsic-size` e dimensiunea estimata – previne *layout shift* cand secțiunea devine vizibilă și se randează real.

Service Worker – cache offline (teaser)

Service Worker este un script care rulează în fundal și poate intercepta cererile HTTP – îl folosești pentru a servi fișiere din cache, oferind funcționalitate **offline**. Combinat cu `manifest.json`, transformă un site în *Progressive Web App* (PWA): instalabil pe telefon, deschis fără internet.

Setup minimal: 30 de linii de JS. Tutorial complet: <https://web.dev/learn/pwa>.

Pentru un portofoliu, valoarea e limitată; pentru un blog citit recurent, sau o aplicație de productivitate, e diferența între „site” și „app”.

Lighthouse CI – audit automat la fiecare commit

GitHub Actions poate rula Lighthouse la fiecare push și **blochează merge-ul** dacă scorul scade sub un prag (ex. 90). Așa nu mai poți accidental regresa.

Listing 50: `.github/workflows/lighthouse.yml` minimal

```
name: Lighthouse CI
on: [pull_request]
jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: treosh/lighthouse-ci-action@v11
        with:
          urls: 'https://voluntar-best.github.io/'
          uploadArtifacts: true
```

Setup complet: <https://github.com/treosh/lighthouse-ci-action>.

Unde mergi de aici

- *web.dev Learn Performance* (curs complet): <https://web.dev/learn/performance>
- *web.dev Fast load times patterns*: <https://web.dev/explore/fast>
- *Bundle size analyzer* (Webpack/Vite): <https://bundlephobia.com>
- *Third-party impact*: <https://www.thirdpartyweb.today>

Verificare cunoștințe

☰ PROVOCARE

1. Care sunt cele 4 categorii evaluate de Lighthouse?
2. Diferența între *lab data* și *field data*?
3. Care sunt cele 3 metrice Core Web Vitals și pragurile lor?
4. *Spot the issue*: `` pe o pagina de productie.
5. Cand este corect sa folosești `loading="lazy"` și cand nu?
6. De ce `font-display: swap` previne „Flash of Invisible Text”?
7. Ce diferență este între `defer` și `async` pe un `<script>`?

Recapitulare

✔ FAZA 6 – LIGHTHOUSE

- Patru categorii: Performance, Accessibility, Best Practices, SEO.
- Praguri: 90+ acceptabil, 95+ profesional, sub 50 necesita interventie.
- Lab data (Lighthouse) vs field data (CrUX); PageSpeed Insights le combina.
- Executie: <https://pagespeed.web.dev> (web) sau Chrome DevTools → *Lighthouse*.
- Top fix-uri: `alt` pe imagini, `lang` pe `<html>`, contrast $\geq 4.5:1$, `width/height` pe `<img>`, `defer` pe `<script>`.
- Core Web Vitals: LCP $< 2.5s$, INP $< 200ms$, CLS < 0.1 .
- Imagini: WebP/AVIF, `srcset`, `loading="lazy"` (cu exceptia imaginii hero), dimensiuni explicite.
- Squoosh: drag → resize → format WebP → quality 75 → download. 500KB → 60KB tipic.
- Fonturi: `font-display: swap` previne FOIT; `<link rel="preload">` pentru fontul critic; self-host pentru „o conexiune mai puțin”.
- CRP: CSS blocheaza render; JS fara `defer` blocheaza tot. `defer` = default pentru JS-ul tau.
- 🦉 Bonus: minification, lazy third-party, `content-visibility`, Service Worker (PWA), Lighthouse CI.



🛠 APLICAȚIE PRACTICĂ

Platforma: PageSpeed Insights (<https://pagespeed.web.dev>).

Activitatea.

1. *Pasul 1 (3 minute)*. Lipirea URL-ului paginii proprii deployate. Executarea analizei. Scorul tipic, dupa parcurgerea fazelor 1–5, este peste 90.
2. *Pasul 2 (5 minute)*. Identificarea uneia sau a doua probleme cu impact ridicat. Aplicarea fix-ului in cod local. `git push`. Reexecutarea analizei dupa 60 de secunde.
3. *Pasul 3 (2 minute)*. Aplicarea aceluasi audit pe varianta deployata a folder-ului `kit-starter/01-broken/`. Compararea scorurilor.

Continuare independenta: folder-ul `01-broken` contine 10 probleme documentate in fisierul README. Tinta este atingerea scorului 95+ pe toate cele patru categorii.

Durata estimata: 10 minute.

Resurse pentru aprofundare

- Google. *PageSpeed Insights*. <https://pagespeed.web.dev>
- Google. *Learn Performance*. <https://web.dev/learn/performance>
- Google. *Web Vitals*. <https://web.dev/articles/vitals>
- Google. *Lighthouse documentation*. <https://developer.chrome.com/docs/lighthouse/overview>
- WebAIM. *Contrast Checker*. <https://webaim.org/resources/contrastchecker/>
- Google. *Squoosh image optimizer*. <https://squoosh.app>
- Real Favicon Generator. <https://realfavicongenerator.net>

→ **Urmeaza:** Capitolul Resurse & urmatorii pași (cap. 7) propune un roadmap de 4 saptamani pentru consolidare și o lista curatata de tutoriale, comunitați și tool-uri pentru a continua singur.

Resurse & urmatorii pasi

Aplicatii interactive. Tutoriale. Referinte. Comunitati.

Acest capitol consolideaza resursele referentiate in capitolele anterioare si adauga sugestii pentru continuarea studiului dupa training. Materialele sunt grupate functional: aplicatii interactive (joculete), tutoriale structurate, referinte de cautare, comunitati, si tool-uri practice.

1. Roadmap pentru consolidare

📌 SAPTAMANA 1

Reconstruirea site-ului cu o tema personala (portofoliu, blog tematic, pagina pentru un eveniment). Folder-ul `kit-starter/02-reference/` serveste ca exemplu de implementare, nu ca sablon copiat direct.

📌 SAPTAMANA 2

Atingerea scorului 95+ pe toate cele patru categorii Lighthouse. Iteratia consta in audit – identificare a problemelor – remediere – reaudit.

📌 SAPTAMANA 3–4

Adaugarea unei functionalitati non-triviale: comutator pentru tema dark/light, formular cu trimitere efectiva (<https://formspring.io> ofera un endpoint gratuit), galerie de imagini cu lightbox, navigare cu mai multe pagini, animatii la scroll.

📌 LUNA 2 SI URMATOARELE

Inscrierea intr-un curriculum complet: *The Odin Project* sau *freeCodeCamp*. Ambele sunt gratuite si acopera traiectoria completa pana la dezvoltare full-stack, in 6–12 luni de studiu sustinut.

2. 🎮 Aplicatii interactive (joculete)

HTML / Accesibilitate

W3C Before / After Demo – comparatie intre o pagina inaccesibila si echivalentul sau accesibil.

CSS

Flexbox Froggy – <https://flexboxfroggy.com>

Grid Garden – <https://cssgridgarden.com>

CSS Diner – <https://flukeout.github.io> (selectori)

CSS Battle – <https://cssbattle.dev> (reproducerea unei imagini cu CSS minimal)

JavaScript

JS Robot – <https://lifesphere.eu/jsrobot>

Coding Fantasy – <https://codingfantasy.com> (lectiile JS sunt gratuite)

Codewars – <https://codewars.com> (probleme de tip kata)

Git / Deploy

Learn Git Branching – <https://learngitbranching.js.org>

Oh My Git! – <https://ohmygit.org> (joc desktop, gratuit, open source)

3. 📖 Tutoriale structurate

- **The Odin Project.** <https://www.theodinproject.com>
Curriculum complet, gratuit, parcurs estimat 6-12 luni. Acoperire: HTML, CSS, JavaScript, Git, Node.js, Ruby on Rails sau React.
- **freeCodeCamp.** <https://www.freecodecamp.org>
Certificari gratuite (Responsive Web Design, JavaScript Algorithms and Data Structures, Front End Development Libraries).
- **web.dev/learn.** <https://web.dev/learn>
Cursuri scurte produse de Google, actualizate frecvent: HTML, CSS, JavaScript, Performance, Forms, Accessibility, PWA.
- **javascript.info.** <https://javascript.info>
Manual extins pentru JavaScript, mai detaliat decat MDN, structurat didactic.
- **W3Schools.** <https://www.w3schools.com>
Referința sistematică cu „Try it Yourself” pe fiecare exemplu. Secțiuni separate: HTML, CSS, JS, TypeScript, React, SQL, Python.
- **Educative – Web Development: Unraveling HTML, CSS, JS.** <https://www.educative.io>
Curs comprehensiv (20h) cu pattern-ul „Hands-On / How it Works” – prezintă acțiunea, apoi explică.
- **Educative – HTML5 Canvas: From Noob to Ninja.** <https://www.educative.io>
Continuare pentru cei interesați de animații și grafica programatică (9h, peste 100 lecții).

4. 🏆 Pregătire pentru interviuri

- **GreatFrontend – 50 must-know HTML/CSS/JS interview questions.**
<https://www.greatfrontend.com/blog/50-must-know-html-css-and-javascript-interview-questions>
Lista celor mai frecvente întrebări tehnice la interviurile front-end, formulată de foști intervieviatori.
- **Frontend Interview Handbook.** <https://www.frontendinterviewhandbook.com>
Resursa free pentru pregătirea interviurilor front-end (sistem design, coding, comportamente).

- **LeetCode – JavaScript track.** <https://leetcode.com>
Probleme algoritmice si de structuri de date in JavaScript.

5. 🔍 Referinte de cautare

- **Mozilla Developer Network.** <https://developer.mozilla.org>
Sursa de referinta pentru standardele web. Cautarea se efectueaza cu pattern-ul „X mdn” pe orice motor de cautare.
- **Can I Use.** <https://caniuse.com>
Tabel detaliat pe browser si versiune pentru suportul fiecarui feature web.
- **CSS-Tricks.** <https://css-tricks.com>
Articole de detaliu pentru concepte CSS specifice; ghidul de flexbox si cel de grid sunt referinte standard.

6. 👥 Comunitati

- Subreddit-urile *r/webdev*, *r/learnprogramming*, *r/Frontend* – discutii zilnice si raspunsuri la intrebari concrete.
- **Stack Overflow.** <https://stackoverflow.com> – platforma de intrebari/raspunsuri tehnice. Primul rezultat pentru majoritatea erorilor cautate pe Google.
- **Frontend Mentor.** <https://www.frontendmentor.io> – provocari de tip design → implementare, cu peer review.
- Canalele Discord ale comunitatii BEST Iasi – pentru intrebari specifice contextului local.

7. 🛠 Tool-uri practice

- **Squoosh** – <https://squoosh.app> (optimizare imagini).
- **TinyPNG** – <https://tinypng.com> (compresie PNG / JPG).
- **Coolers** – <https://coolers.co> (generare palete de culori).
- **Google Fonts** – <https://fonts.google.com>.
- **Favicon Generator** – <https://favicon.io>.
- **Excalidraw** – <https://excalidraw.com> (diagrame, schite tehnice).
- **Cubic-Bezier editor** – <https://cubic-bezier.com> (curbe de easing personalizate).
- **Clippy** – <https://bennettfeely.com/clippy/> (clip-path vizual).
- **Google Webfonts Helper** – <https://gwfh.mranftl.com> (self-hosting fonturi cu font-display).
- **Schema.org generator** – <https://technicalseo.com/tools/schema-markup-generator/> (JSON-LD vizual).
- **XML Sitemap generator** – <https://www.xml-sitemaps.com>.

8. 🦉 Topici avansate – unde țintim dupa training

Secțiunile *Bonus / Avansat* din capitolele 1–6 introduc concepte care merita aprofundare independenta. Mai jos, link-urile principale pentru fiecare:

- **CSS animations & 3D.**

<https://web.dev/learn/css/animations> – cursul complet

<https://developer.chrome.com/docs/css-ui/scroll-driven-animations> – animation-timeline

- **JavaScript – pattern-uri practice.**

<https://javascript.info/async> – promises, async/await

https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API – IntersectionObserver

<https://css-tricks.com/debouncing-throttling-explained-examples/> – debounce/-throttle

- **Performance avansat.**

<https://web.dev/learn/performance> – curs complet

<https://www.thirdpartyweb.today> – impactul scripturilor de la alții

<https://github.com/treosh/lighthouse-ci-action> – audit automat in GitHub Actions

- **PWA și offline.**

<https://web.dev/learn/pwa> – Service Worker + Manifest, de la zero

- **Free public APIs (pentru exersat `fetch`).**

<https://github.com/public-apis/public-apis>

9. 🧠 Idei de proiecte (pentru cand te plictisești)

Cele mai bune skill-uri se invata facand ceva ce te intereseaza pe tine. Cateva idei, de la 30 minute la cateva weekenduri:

Personale (pentru CV / share pe social)

- Portofoliu personal cu poza, sectiune „Despre”, proiecte, contact.
- Blog despre orice te intereseaza (gaming, anime, gatit, drumetii, fotografii).
- „Card de vizita” digital (CV web stylish, share-uibil prin URL).
- Pagina pentru un proiect BEST sau un eveniment universitar.

Distractive (1-2 ore fiecare)

- Tic-Tac-Toe sau Memory game in HTML/CSS/JS pur.
- Quiz interactiv cu intrebari preferate („Ce film e asta?”, „Ghicește tara dupa steag”).
- Pomodoro timer (25 min lucru / 5 min pauza, cu sunet).
- Calculator simplu, dar cu un design mișto.
- Generator de palete de culori la apasare de buton.
- Joc snake clasic, in canvas.

Utile (le folosești tu, prietenii tai)

- Habit tracker (verde la zilele cand ai facut treaba, roșu la cand n-ai).
- Lista de filme / serii / cărți de vazut, cu rating.
- Tracker de cheltuieli (carbohidrați, bani, ore studiate).
- Pagina pentru proiectul tau de școală (mai stylish decat un PDF).

Fan / pasiune

- Pagina dedicata serialului / anime-ului / formației tale preferate.
- Galerie cu pozele tale de la concerte / drumeții / vacanțe.
- Wiki personal pentru jocul tau favorit (Skyrim, Pokemon, etc.).

10. 🏆 Provocari pentru cand vrei sa te depășești

- „Site in 1 ora”. Cronometreaza-te. Pune un site decent online intr-o ora.
- „Lighthouse 100 / 100 / 100 / 100”. Niciun fix nu este de prisos.
- „30 zile de cod”. Un commit pe zi, orice marime, pe orice repo. Vezi cum se umple GitHub-ul de patrate verzi.
- „No CSS framework”. Refa unul dintre proiectele tale fara Bootstrap/Tailwind – doar CSS pur.
- „Cont GitHub cu 5 stele”. Construiește ceva de care alții vor sa puna stea. (Nu le cere stele – fa-le de la sine.)



✔ OBSERVATIE DE INCHEIERE

Slide-urile prezentarii si pagina personala construita in training partajeaza acelasi set de tehnologii (HTML, CSS, JavaScript). Functionalitatea *View Source* a oricarui browser ofera acces la codul sursa al oricarei pagini publice. Lectura acelui cod este, de acum inainte, accesibila.

Anexe

Glosar termeni. Checklist final. Erori comune si remediere.

Sectiunea finala consolideaza referintele rapide: definitiile termenilor folositi pe parcursul manualului, lista de verificari inainte de publicarea unui site, si problemele intalnite frecvent impreuna cu solutiile lor.

1. Glosar de termeni

Accesibilitate (a11y)

Practica de a crea continut utilizabil de catre persoane cu dizabilitati. Standardul de referinta este WCAG (Web Content Accessibility Guidelines).

API

Application Programming Interface. Set de functii prin care componentele software interactioneaza. In context web: `document.querySelector`, `fetch`, etc.

Arrow function

Sintaxa concisa pentru functii in JavaScript: `(a, b) => a + b`. Se comporta diferit fata de `function` privind `this`.

Cascade (CSS)

Algoritmul prin care browserul determina valoarea finala a unei proprietati cand mai multe reguli o vizeaza. Foloseste specificitate, ordine si importanta.

CDN

Content Delivery Network. Reteaua de servere distribuite geografic pentru servirea resurselor cu latenta minima.

CLS

Cumulative Layout Shift. Indicator Core Web Vitals pentru stabilitatea vizuala in timpul incarcarii.

Commit

Snapshot al starii fisierelor intr-un repository git, identificat printr-un hash unic.

CORS

Cross-Origin Resource Sharing. Mecanism de securitate care controleaza accesul la resurse din alte domenii.

CSS Grid

Model de layout bidimensional, complementar lui Flexbox; util pentru aranjamente in retea.

Cubic-bezier

Funcție de timing CSS care definește o curba de accelerare prin patru numere (doua puncte de control), pentru animații personalizate.

Debounce

Tehnica JS care amana executia unei funcții pana cand evenimentul nu se mai declanșează N ms. Folosit pentru input/autocomplete.

DOM

Document Object Model. Reprezentarea in memorie a documentului HTML, expusa programatic prin `document`.

Endpoint

URL specific care accepta cereri HTTP si returneaza un raspuns. Echivalent cu o „adresa” a unui serviciu.

Flexbox

Model de layout uni-dimensional in CSS pentru distribuirea spatiului pe o axa.

Font-display

Proprietate `@font-face` care controleaza ce vede utilizatorul cat timp fontul se descarca. `swap` = arata fallback imediat, evita FOIT.

Hosting

Servirea fisierelor unui site catre vizitatori. GitHub Pages, Netlify si Vercel sunt servicii de hosting pentru site-uri statice.

HTTP

HyperText Transfer Protocol. Protocolul de baza al webului, prin care browserele cer si primesc resurse.

INP

Interaction to Next Paint. Indicator Core Web Vitals pentru receptivitate; latentă între interacțiune și feedback vizual.

IntersectionObserver

API browser care notifica un callback cand un element intra/iese din viewport. Inlocuieste handler-urile de scroll pentru animații și lazy loading.

JSON-LD

JSON for Linked Data. Format de date structurate folosit de motoarele de cautare pentru intelegerea continutului.

LCP

Largest Contentful Paint. Indicator Core Web Vitals; momentul afisarii celui mai mare element vizibil.

Lighthouse

Tool de audit automatizat dezvoltat de Google; evalueaza Performance, Accessibility, Best Practices, SEO.

NodeList

Colectie de noduri DOM returnata de `querySelectorAll`; iterabila prin `forEach`.

Open Graph (OGP)

Standard inițiat de Facebook pentru previzualizarea paginilor pe rețele sociale (`og:title`, `og:image`, etc.).

PAT

Personal Access Token. Token de autentificare pentru GitHub, folosit în locul parolei la operațiunile git.

Preconnect

Hint HTML (`<link rel="preconnect">`) care instruieste browserul sa stabileasca conexiune (DNS + TCP + TLS) cu un domeniu in avans, reducand latenta pentru prima cerere.

Pseudo-clasa

Selector CSS care vizeaza o stare a unui element: `:hover`, `:focus`, `:checked`, `:first-child`.

Repository (repo)

Director versionat de git; contine codul, istoricul commit-urilor si configuratia.

`requestAnimationFrame`

API browser care apeleaza un callback inainte de fiecare repaint (~60 fps). Inlocuieste `setTimeout` pentru animații JavaScript fluide.

Responsive design

Abordare prin care site-ul se adapteaza dimensiunii ecranului utilizatorului.

`robots.txt`

Fisier text in radacina site-ului care indica robotilor de crawl ce pagini sa indexeze și unde se gasește sitemap-ul.

Schema.org

Vocabular standardizat pentru date structurate, suportat de motoarele de cautare majore.

Selector CSS

Pattern care identifica elementele HTML asupra carora se aplica o regula CSS.

Semantic HTML

Folosirea tag-urilor care descriu rolul continutului (`<header>`, `<nav>`) in locul `<div>` generic.

Service Worker

Script care ruleaza in fundalul browserului, poate intercepta cererile HTTP. Baza pentru PWA și funcționalitate offline.

`sitemap.xml`

Fisier XML care listeaza URL-urile importante ale site-ului. Submis in Google Search Console pentru indexare mai rapida.

Specificitate

Mecanism prin care browserul rezolva conflictele intre reguli CSS care vizeaza acelasi element.

Static site

Site format din fișiere HTML/CSS/JS pre-generate, servite direct, fara procesare pe server.

Throttle

Tehnica JS care permite executia unei funcții cel mult o data la N ms. Folosit pentru `scroll`, `resize`.

Viewport

Zona vizibila a paginii in browser. Meta tag-ul `viewport` controleaza scalarea pe dispozitive mobile.

WCAG

Web Content Accessibility Guidelines. Standardul international pentru accesibilitatea continutului web.

2. Checklist de publicare

Lista de verificari aplicabila inainte de a considera un site finalizat. Se parcurge de sus in jos; orice item nebifat indica o lacuna remediabila.

HTML – structura

- ☐ Documentul incepe cu `<!DOCTYPE html>`.
- ☐ `<html>` are atributul `lang`.
- ☐ `<head>` include `<meta charset>`, `<meta viewport>`, `<title>` specific paginii.
- ☐ Structura folosesc tag-uri semantice (`<header>`, `<main>`, `<section>`, `<footer>`, `<nav>`).
- ☐ Un singur `<h1>` per pagina; ierarhia titlurilor nu sare niveluri.
- ☐ Toate imaginile au `alt`; cele decorative au `alt=""`.
- ☐ Toate input-urile au `<label for>` corespunzator.

CSS – aspect

- ☐ Reset minimal aplicat (`box-sizing: border-box, margin/padding default 0`).
- ☐ Variabilele CSS folosite pentru culori, spatiere, dimensiuni.
- ☐ Layout-ul rezista pe ecran 320px (test in DevTools – mobile preview).
- ☐ Media queries pentru ecrane ≥ 768 px.
- ☐ Contrast text/background $\geq 4.5:1$ pentru text normal.

JavaScript – comportament

- ☐ `<script>` are atributul `defer`.
- ☐ Niciun `var`; doar `const` si `let`.
- ☐ Comparatii doar cu `===`.
- ☐ Console-ul (F12) nu afiseaza erori la incarcarea paginii.

- ☐ Form-urile interceptate cu `e.preventDefault()` (sau au `action valid`).

SEO + Open Graph

- ☐ `<title>` specific, 50–60 caractere.
- ☐ `<meta name="description">` 150–160 caractere.
- ☐ Cinci tag-uri `og:*`: type, title, description, image, url.
- ☐ `og:image` este URL absolut, dimensiune $\geq 1200 \times 630$.
- ☐ `<meta name="twitter:card" content="summary_large_image">` prezent.
- ☐ `<link rel="canonical">` catre URL-ul oficial.
- ☐ `robots.txt` prezent in radacina, cu directiva `Sitemap:`.
- ☐ `sitemap.xml` prezent in radacina; trimis in Google Search Console.
- ☐ Favicon (`<link rel="icon">`) configurat.

Deploy + Lighthouse

- ☐ Site accesibil pe URL public (`username.github.io/repo`).
- ☐ `.gitignore` prezent in repository.
- ☐ Imagini convertite in WebP / AVIF, redimensionate la latimea de afișare.
- ☐ `<img>` are `width`, `height`, `loading="lazy"` (excepție: hero).
- ☐ Fonturi cu `font-display: swap`; fontul critic e `preload`-at.
- ☐ Lighthouse Performance ≥ 90 .
- ☐ Lighthouse Accessibility ≥ 95 .
- ☐ Lighthouse Best Practices ≥ 95 .
- ☐ Lighthouse SEO ≥ 95 .
- ☐ Previzualizare validata pe `opengraph.xyz`.

3. Erori comune si remediere

Lista neexhaustiva de probleme intalnite frecvent de incepatori, cu cauza si solutia.

HTML / CSS

- **Diacriticele apar ca moză.** Cauza: lipsa `<meta charset="UTF-8">` sau fisier salvat in alta codificare. Solutie: adaugarea meta tag-ului si salvarea fisierului in UTF-8 din editor.
- **Stilurile CSS nu se aplica.** Cauze: (1) calea `href` incorecta in `<link>`; (2) numele clasei diferit fata de selector; (3) regula are specificitate mai mica decat alta. Verificare: DevTools → Inspect → tab *Styles*.
- **Pe mobil pagina apare mica/dezordonat.** Cauza: lipsa `<meta name="viewport">`.

- **Flexbox-ul nu functioneaza.** Cauza: `display: flex` aplicat pe element gresit; trebuie pe *containerul* parinte, nu pe copii.

JavaScript

- `querySelector` returns `null`. Cauze: (1) selectorul nu corespunde niciunui element; (2) script-ul ruleaza inainte de parsarea HTML-ului (lipseste `defer`); (3) typo in numele clasei/id-ului.
- `addEventListener` is not a function. Cauza: rezultatul `querySelector` este `null`. Verificare: `console.log(el)` inainte de `addEventListener`.
- **Form-ul reincarca pagina la submit.** Cauza: lipsa `e.preventDefault()` in handler.
- `console.log` not defined. Foarte rar; de obicei o tipografica (`Console.log` cu C mare).

Git / Deploy

- `git push` cere parola si o respinge. Cauza: GitHub a inlocuit autentificarea prin parola cu PAT. Solutie: generare PAT la <https://github.com/settings/tokens> cu scope `repo`; folosirea tokenului in locul parolei.
- **Pages returneaza 404 dupa push.** Cauze: (1) intervalul de propagare (~60s); (2) fisierul de pornire nu se numeste `index.html` (case-sensitive); (3) cache-ul browserului – `Ctrl+F5` pentru hard refresh.
- `fatal: not a git repository`. Cauza: comanda rulata in afara unui director versionat. Solutie: `cd` in directorul corect sau `git init -b main`.
- **Modificarile nu apar pe Pages.** Cauza: nu s-a executat `git push` dupa `commit`; sau cache-ul Pages – asteptare ~60s.

SEO + Lighthouse

- **Card-ul WhatsApp nu afiseaza imaginea.** Cauza principala: `og:image` are URL relativ in loc de absolut. Verificare: opengraph.xyz raporteaza statusul.
- **Lighthouse Performance scor mic.** Cauze frecvente: (1) imagini nedimensionate, supradimensionate; (2) script-uri sincrone in `<head>`; (3) format imagini neoptimizat (PNG/JPEG cu fisiere mari).
- **Lighthouse Accessibility 50–80.** Cauze frecvente: (1) `alt` lipsa; (2) contrast scazut; (3) lipsa `lang` pe `<html>`; (4) butoane fara nume accesibil.